

Binary Classification

Erisa Terolli

Slide Credits to Shusen Wang

Binary Classification

Tasks

Methods

Algorithms

Binary Classification

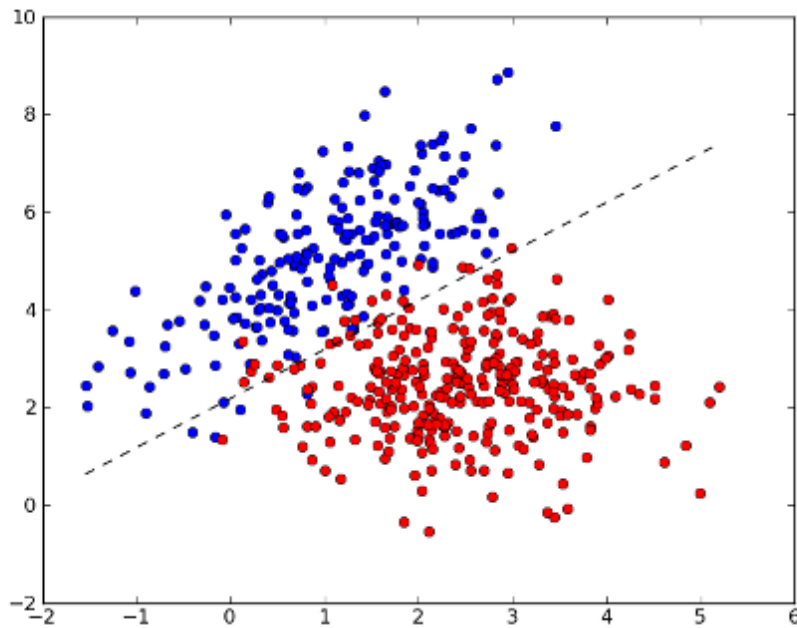
Input: feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $y_1, \dots, y_n \in \{-1, +1\}$.

Output: a function $f: \mathbb{R}^d \mapsto \{-1, +1\}$.

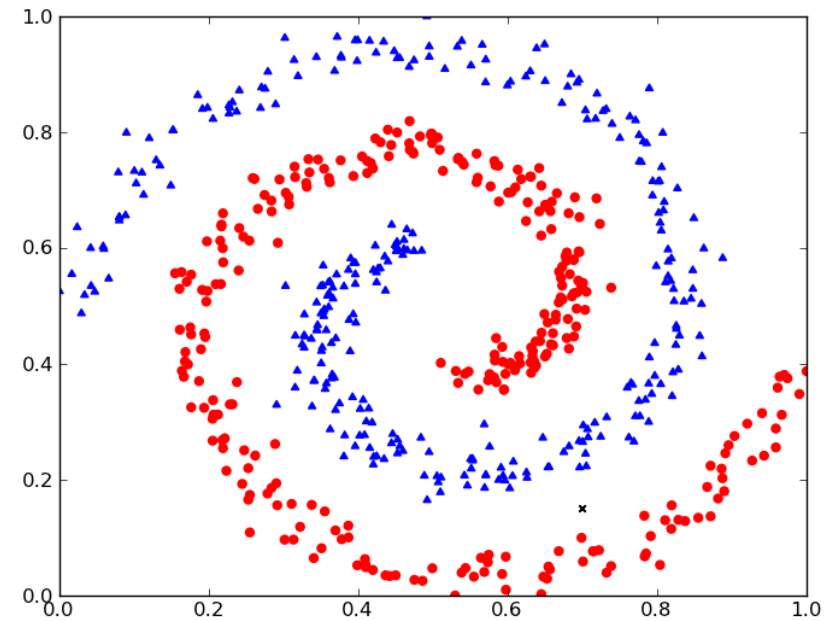
Binary Classification

Input: feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $y_1, \dots, y_n \in \{-1, +1\}$.

Output: a function $f: \mathbb{R}^d \mapsto \{-1, +1\}$.



Linear Classification



Nonlinear Classification

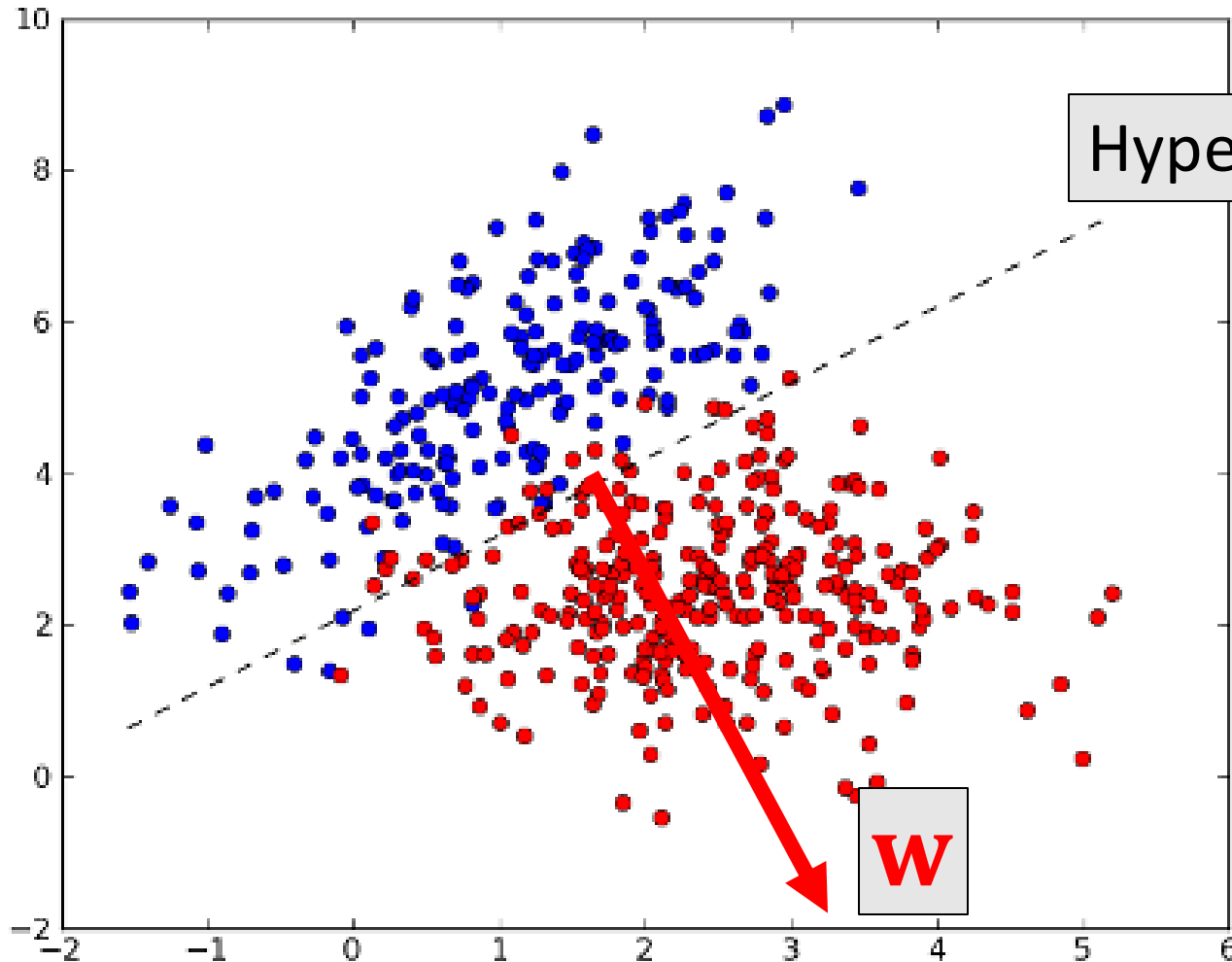
Logistic Regression (Linear Classifier)

Tasks

Methods

Algorithms

Linear Classifier



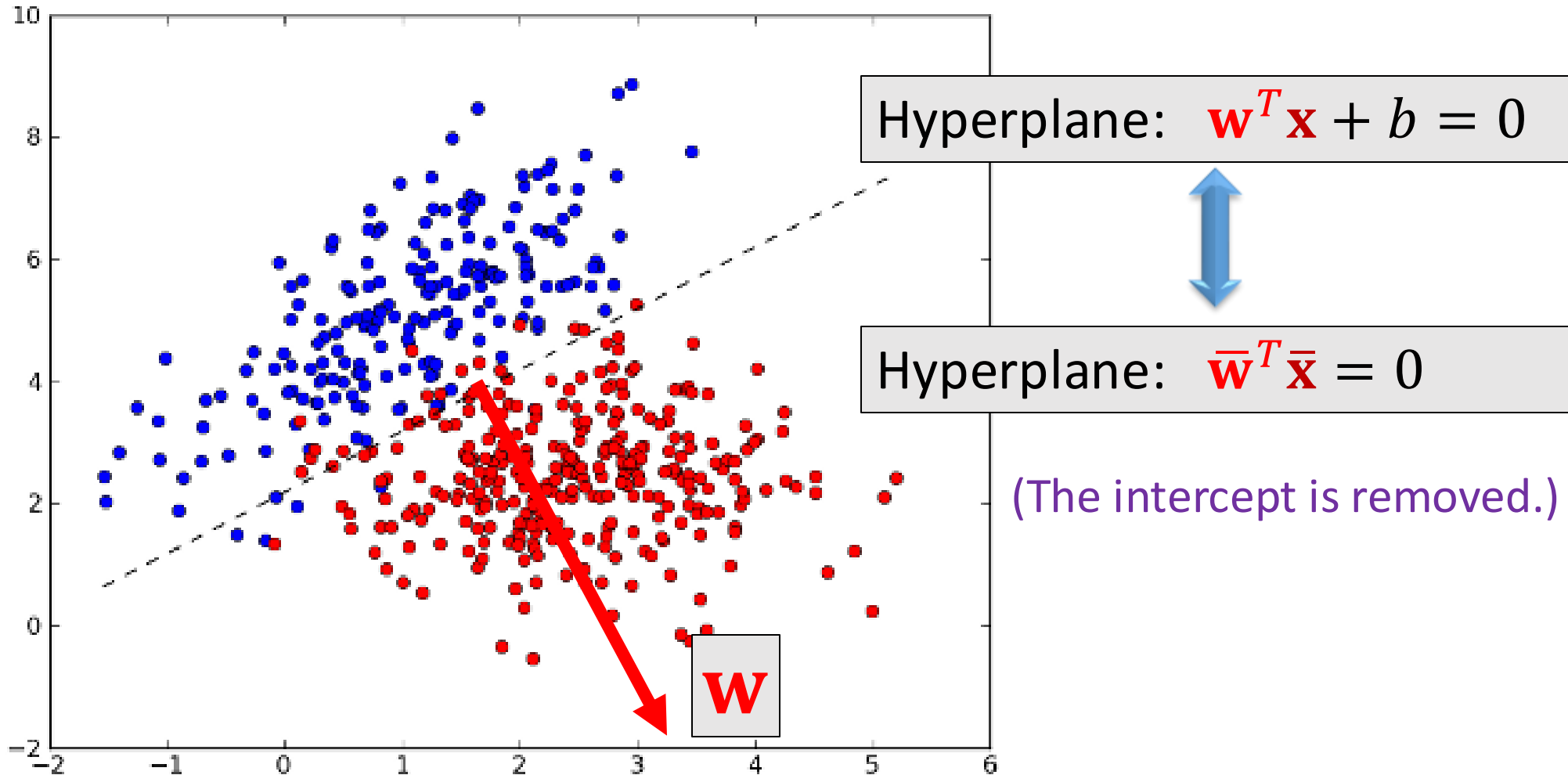
Hyperplane: $\mathbf{w}^T \mathbf{x} + b = 0$

Define $\bar{\mathbf{x}}_j = [\mathbf{x}_j; 1] \in \mathbb{R}^{d+1}$

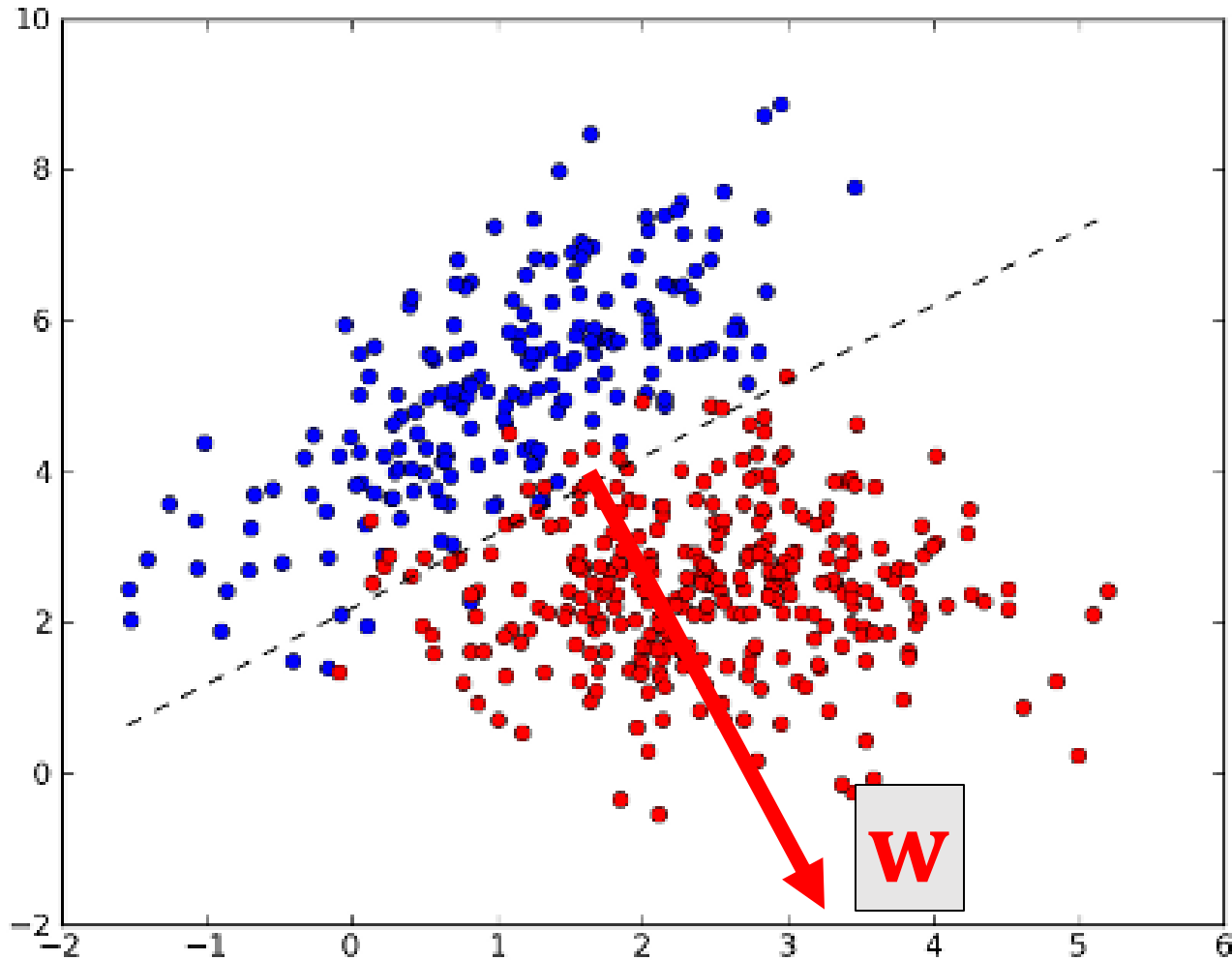
Define $\bar{\mathbf{w}} = [\mathbf{w}; b] \in \mathbb{R}^{d+1}$

$\Rightarrow \mathbf{x}_j^T \mathbf{w} + b = \bar{\mathbf{x}}_j^T \bar{\mathbf{w}}$

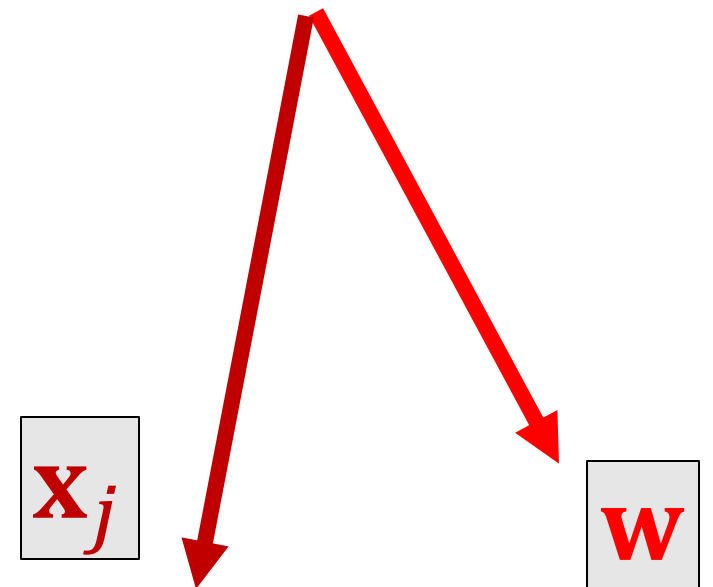
Linear Classifier



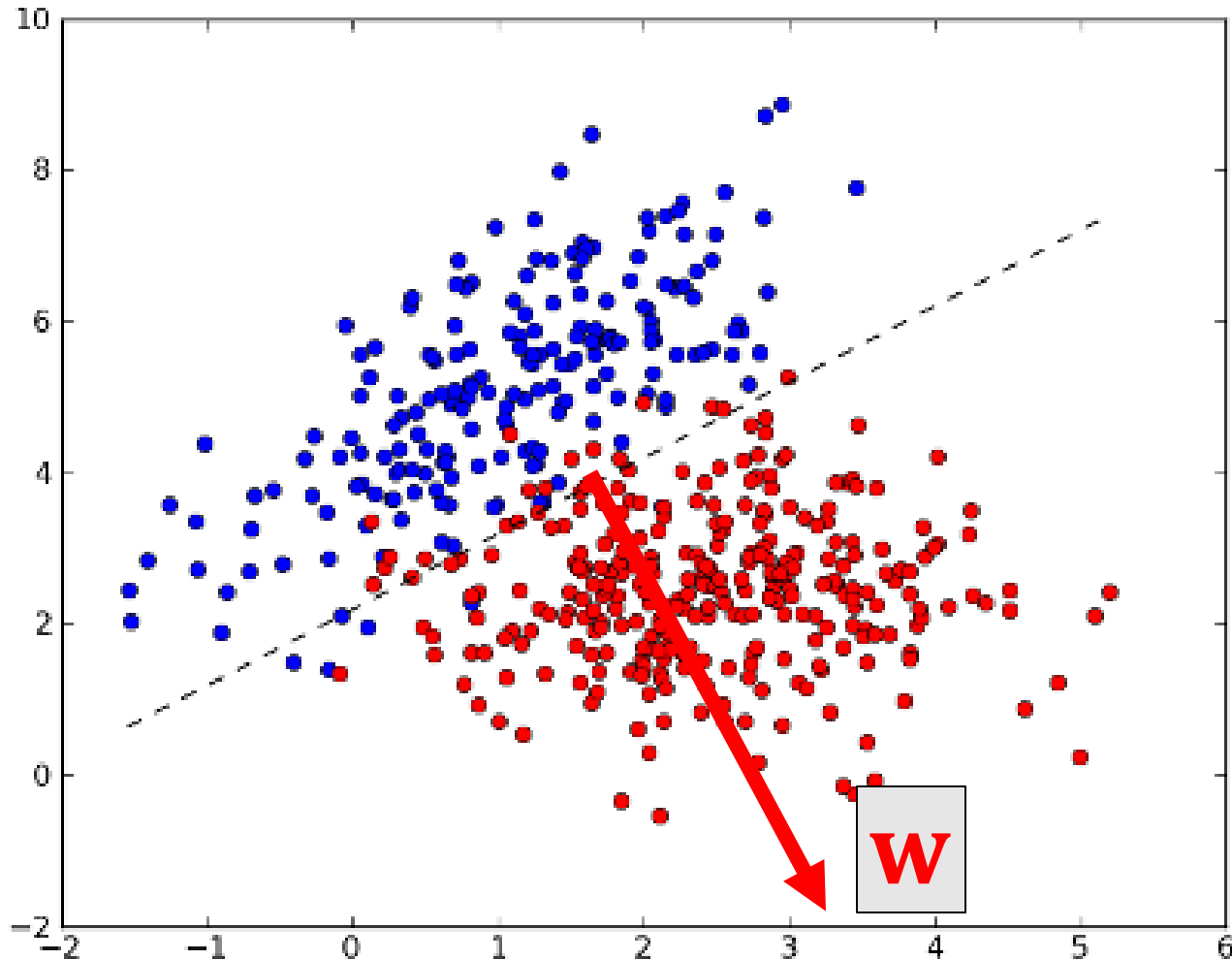
Linear Classifier



- Learn a vector $\mathbf{w}$ such that
- If $y_j = +1$, then $\mathbf{w}^T \mathbf{x}_j > 0$.



Linear Classifier



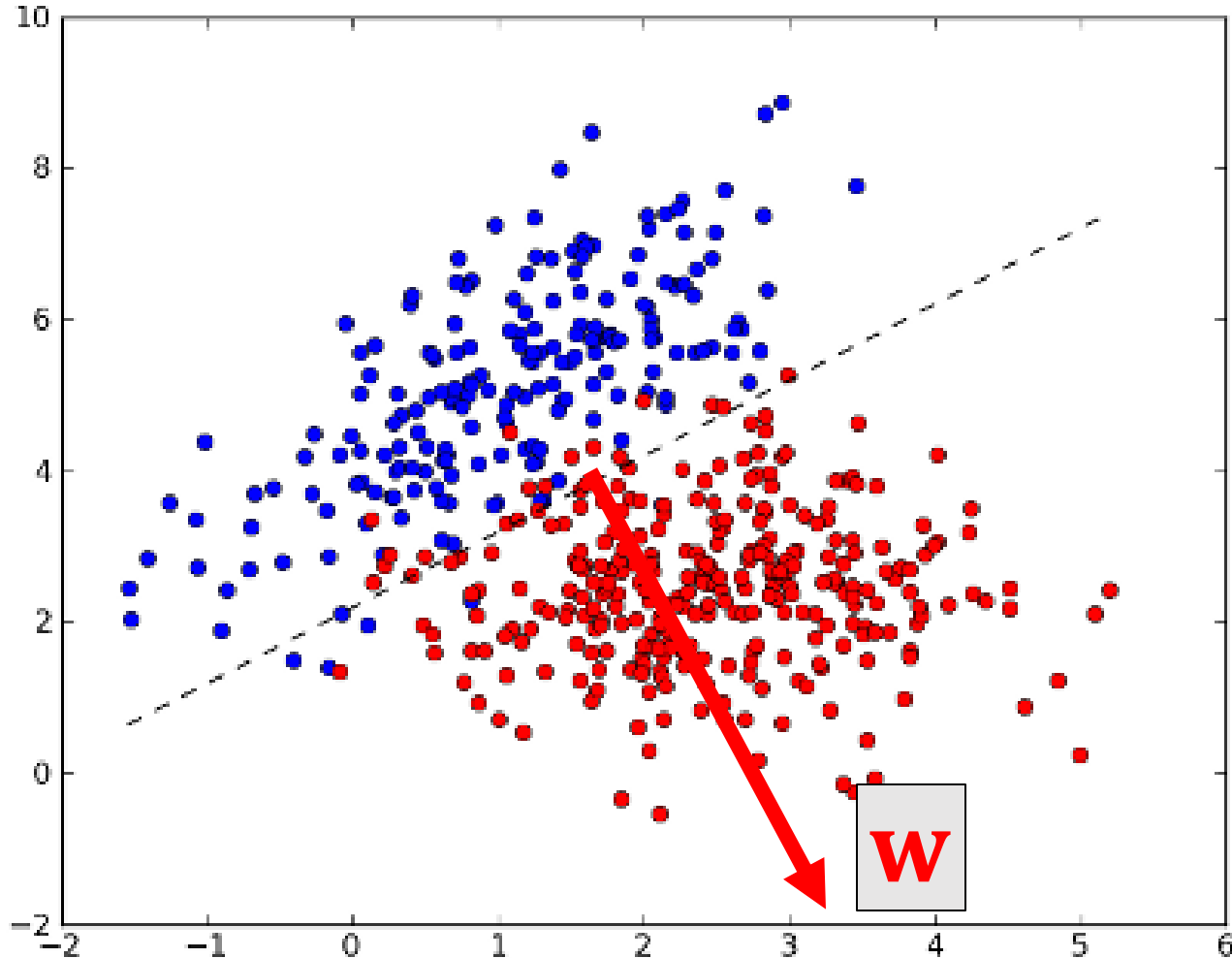
Learn a vector $\mathbf{w}$ such that

- If $y_j = +1$, then $\mathbf{w}^T \mathbf{x}_j > 0$.
- If $y_j = -1$, then $\mathbf{w}^T \mathbf{x}_j < 0$.

$\mathbf{x}_j$

$\mathbf{w}$

Linear Classifier



Learn a vector $\mathbf{w}$ such that

- If $y_j = +1$, then $\mathbf{w}^T \mathbf{x}_j > 0$.
- If $y_j = -1$, then $\mathbf{w}^T \mathbf{x}_j < 0$.

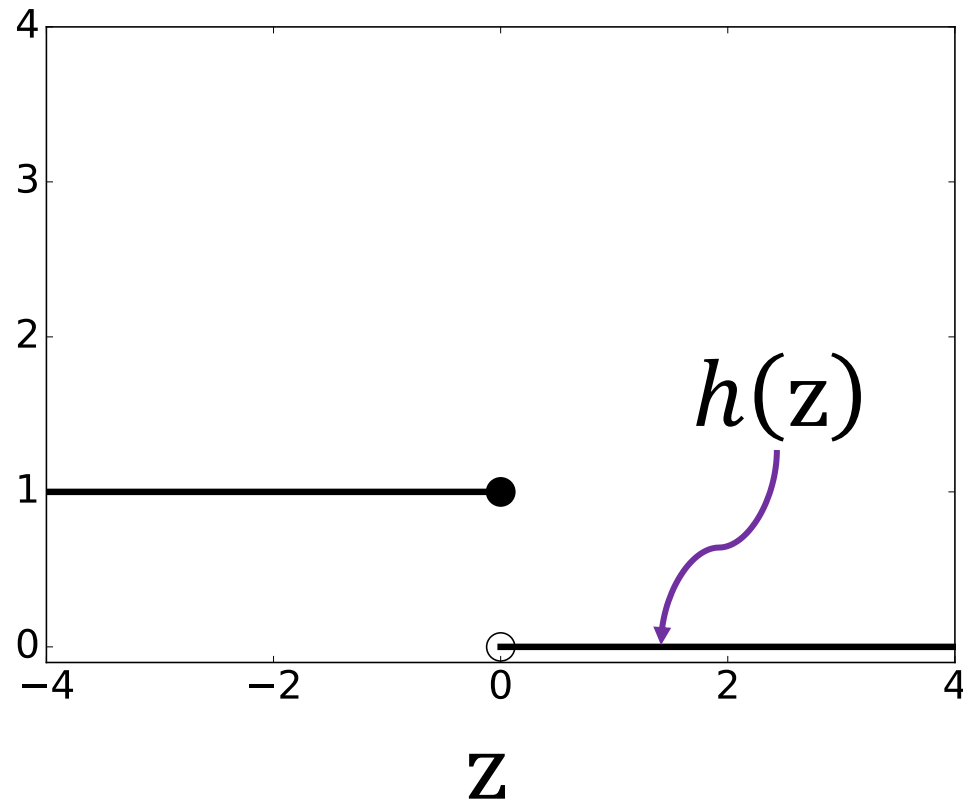


Key Idea:

Encourage $y_j \mathbf{w}^T \mathbf{x}_j$ to be positive

Directly Minimize the Classification Error?

Minimize $\sum_j h(y_j \mathbf{w}^T \mathbf{x}_j)$, where $h(z) = \begin{cases} 1, & \text{if } z < 0; \\ 0, & \text{if } z \geq 0. \end{cases}$

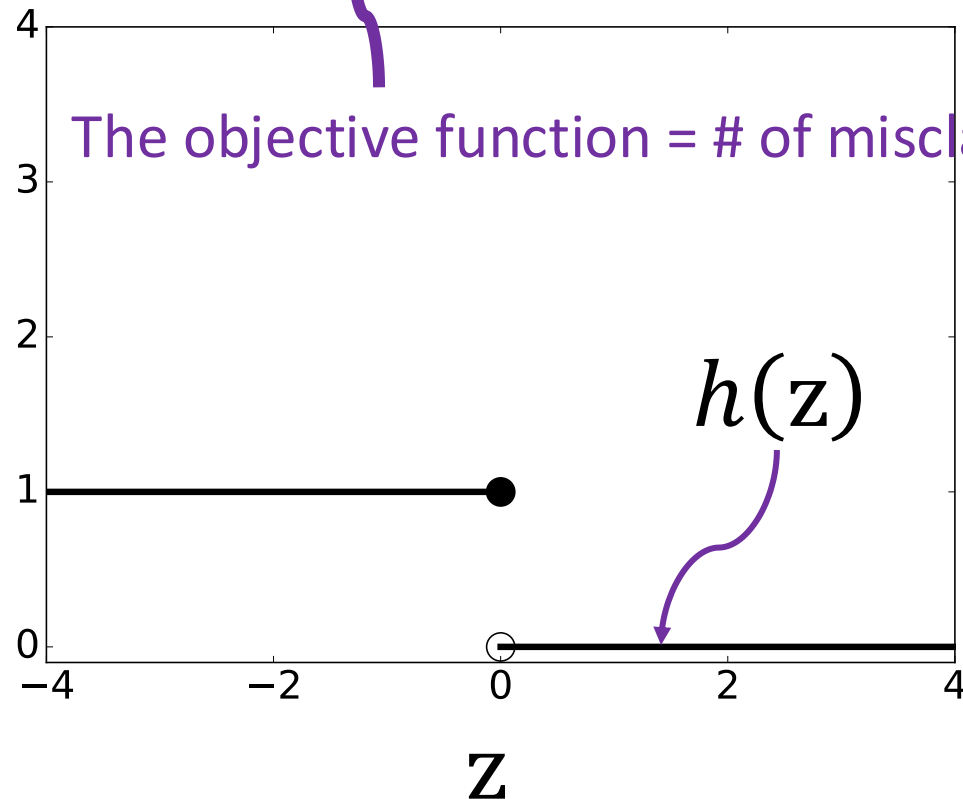


Key Idea:

Encourage $y_j \mathbf{w}^T \mathbf{x}_j$ to be positive

Directly Minimize the Classification Error?

Minimize $\sum_j h(y_j \mathbf{w}^T \mathbf{x}_j)$, where $h(z) = \begin{cases} 1, & \text{if } z < 0; \\ 0, & \text{if } z \geq 0. \end{cases}$



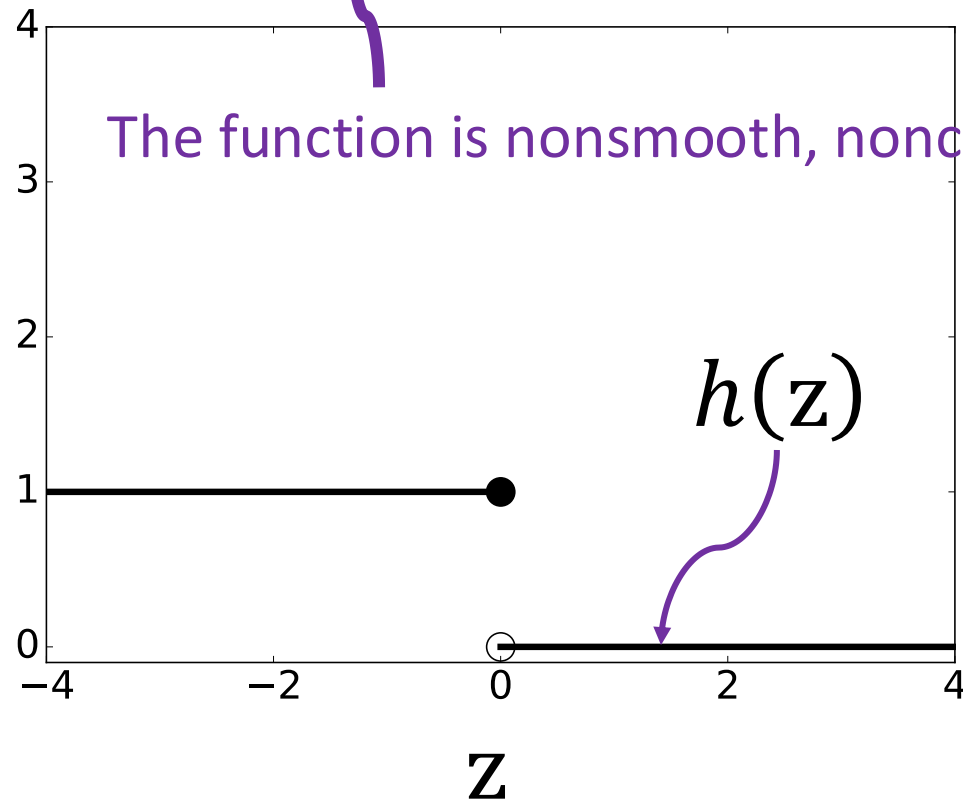
The objective function = # of misclassified training samples

Key Idea:

Encourage $y_j \mathbf{w}^T \mathbf{x}_j$ to be positive

Directly Minimize the Classification Error?

Minimize $\sum_j h(y_j \mathbf{w}^T \mathbf{x}_j)$, where $h(z) = \begin{cases} 1, & \text{if } z < 0; \\ 0, & \text{if } z \geq 0. \end{cases}$



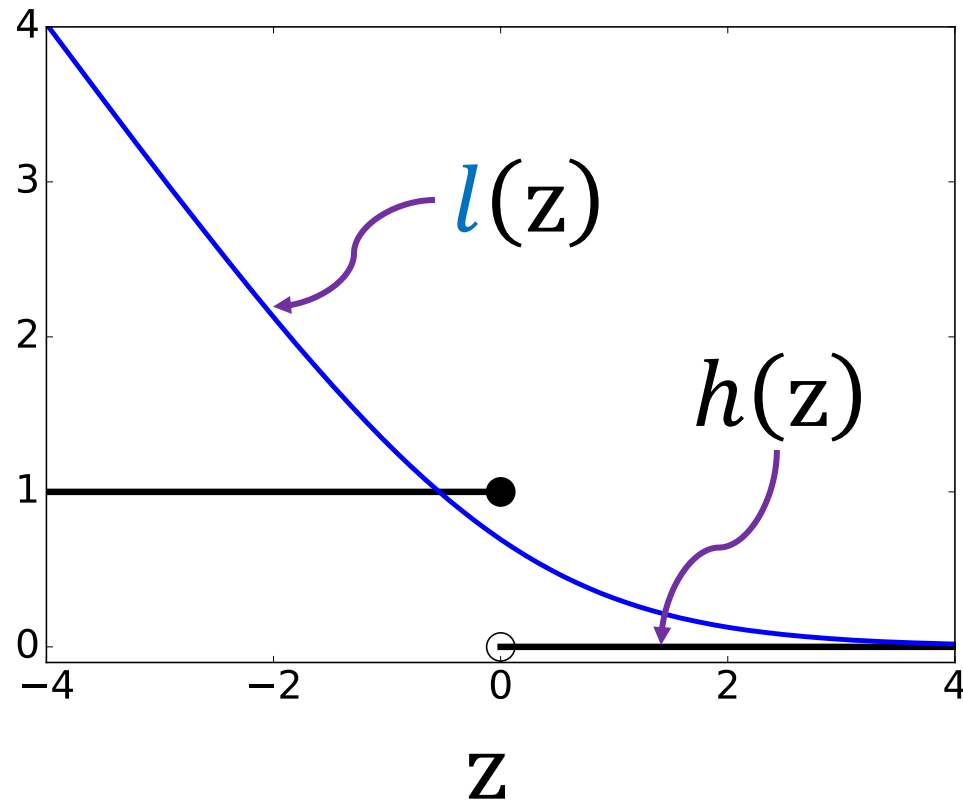
The function is nonsmooth, nonconvex, and hard to optimize.

Key Idea:

Encourage $y_j \mathbf{w}^T \mathbf{x}_j$ to be positive

Logistic Regression

Minimize $\sum_j l(y_j \mathbf{w}^T \mathbf{x}_j)$, where $l(z) = \log(1 + e^{-z})$.



Key Idea:
Encourage $y_j \mathbf{w}^T \mathbf{x}_j$ to be positive

Logistic Regression

Tasks

Methods

Algorithms

Logistic Regression

Logistic regression: $\min_{\mathbf{w}} \frac{1}{n} \sum_{j=1}^n l(y_j \mathbf{w}^T \mathbf{x}_j)$, where $l(z) = \log(1 + e^{-z})$.

Tasks

Binary Classification

Multi-Class Classification

Methods

Logistic Regression

SVM

Neural Networks

Algorithms

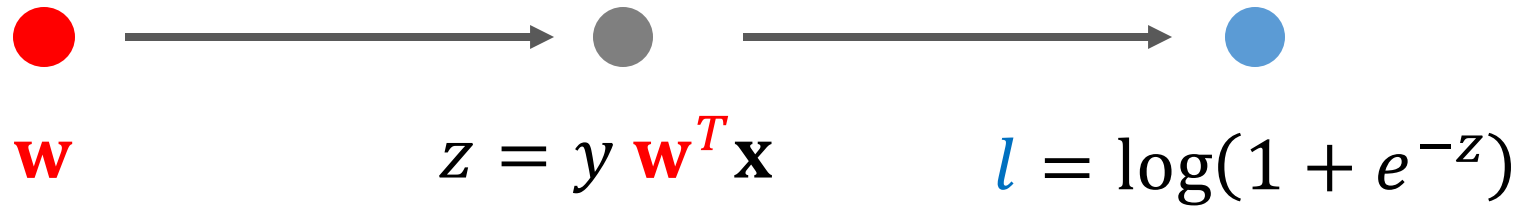
Gradient Descent (GD)

Accelerated GD

Stochastic GD

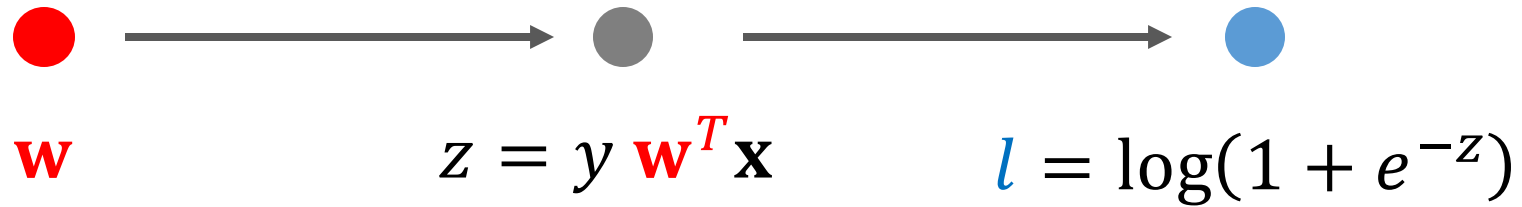
Gradient

Logistic regression: $\min_{\mathbf{w}} \frac{1}{n} \sum_{j=1}^n l(y_j \mathbf{w}^T \mathbf{x}_j)$, where $l(z) = \log(1 + e^{-z})$.



Gradient

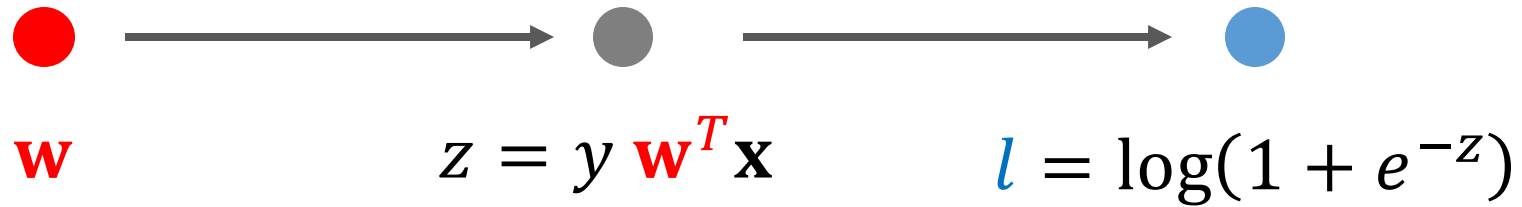
Logistic regression: $\min_{\mathbf{w}} \frac{1}{n} \sum_{j=1}^n l(y_j \mathbf{w}^T \mathbf{x}_j)$, where $l(z) = \log(1 + e^{-z})$.



- $\frac{\partial z}{\partial \mathbf{w}} = y\mathbf{x},$

Gradient

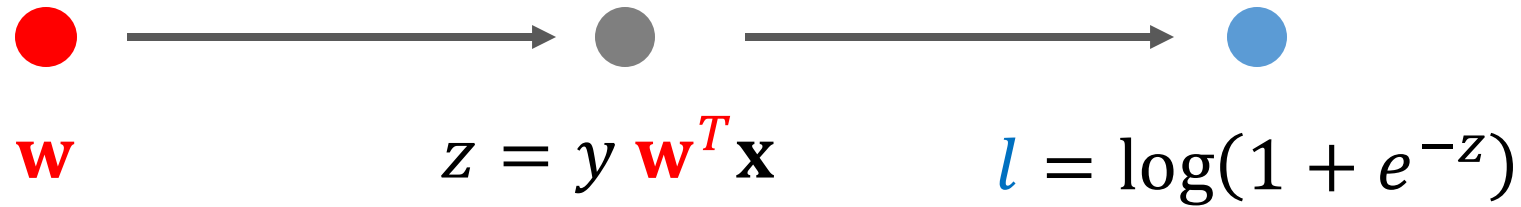
Logistic regression: $\min_{\mathbf{w}} \frac{1}{n} \sum_{j=1}^n l(y_j \mathbf{w}^T \mathbf{x}_j)$, where $l(z) = \log(1 + e^{-z})$.



- $\frac{\partial z}{\partial \mathbf{w}} = y\mathbf{x}, \quad \frac{\partial l(z)}{\partial z} = \frac{-e^{-z}}{1+e^{-z}} = -\frac{1}{1+e^z}.$

Gradient

Logistic regression: $\min_{\mathbf{w}} \frac{1}{n} \sum_{j=1}^n l(y_j \mathbf{w}^T \mathbf{x}_j)$, where $l(z) = \log(1 + e^{-z})$.



- $\frac{\partial z}{\partial \mathbf{w}} = y\mathbf{x}, \quad \frac{\partial l(z)}{\partial z} = \frac{-e^{-z}}{1+e^{-z}} = -\frac{1}{1+e^z}.$
- Chain rule: $\frac{\partial l}{\partial \mathbf{w}} = \frac{\partial z}{\partial \mathbf{w}} \cdot \frac{\partial l}{\partial z} = (y\mathbf{x}) \left(-\frac{1}{1+e^z} \right) = -\frac{y\mathbf{x}}{1+\exp(y\mathbf{w}^T \mathbf{x})}.$

Gradient

Logistic regression: $\min_{\mathbf{w}} \frac{1}{n} \sum_{j=1}^n l(y_j \mathbf{w}^T \mathbf{x}_j)$, where $l(z) = \log(1 + e^{-z})$.

- We have shown: $\frac{\partial l(y \mathbf{w}^T \mathbf{x})}{\partial \mathbf{w}} = \frac{-y \mathbf{x}}{1 + \exp(y \mathbf{w}^T \mathbf{x})}$.
- Objective function: $f(\mathbf{w}) = \frac{1}{n} \sum_j l(y_j \mathbf{w}^T \mathbf{x}_j)$.

Gradient

Logistic regression: $\min_{\mathbf{w}} \frac{1}{n} \sum_{j=1}^n l(y_j \mathbf{w}^T \mathbf{x}_j)$, where $l(z) = \log(1 + e^{-z})$.

- We have shown: $\frac{\partial l(y \mathbf{w}^T \mathbf{x})}{\partial \mathbf{w}} = \frac{-y \mathbf{x}}{1 + \exp(y \mathbf{w}^T \mathbf{x})}$.
- Objective function: $f(\mathbf{w}) = \frac{1}{n} \sum_j l(y_j \mathbf{w}^T \mathbf{x}_j)$.
- $\frac{\partial f(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{n} \sum_j \frac{\partial l(y_j \mathbf{w}^T \mathbf{x}_j)}{\partial \mathbf{w}} = \frac{1}{n} \sum_j \frac{-y_j \mathbf{x}_j}{1 + \exp(y_j \mathbf{w}^T \mathbf{x}_j)}$.

Gradient

Logistic regression: $\min_{\mathbf{w}} \frac{1}{n} \sum_{j=1}^n l(y_j \mathbf{w}^T \mathbf{x}_j)$, where $l(z) = \log(1 + e^{-z})$.

$$\text{Gradient at } \mathbf{w}_t: \mathbf{g}_t = \frac{1}{n} \sum_{j=1}^n \frac{-y_j \mathbf{x}_j}{1 + \exp(y_j \mathbf{w}_t^T \mathbf{x}_j)}.$$

- We have shown: $\frac{\partial l(y \mathbf{w}^T \mathbf{x})}{\partial \mathbf{w}} = \frac{-y \mathbf{x}}{1 + \exp(y \mathbf{w}^T \mathbf{x})}$.
- Objective function: $f(\mathbf{w}) = \frac{1}{n} \sum_j l(y_j \mathbf{w}^T \mathbf{x}_j)$.
- $\frac{\partial f(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{n} \sum_j \frac{\partial l(y_j \mathbf{w}^T \mathbf{x}_j)}{\partial \mathbf{w}} = \frac{1}{n} \sum_j \frac{-y_j \mathbf{x}_j}{1 + \exp(y_j \mathbf{w}^T \mathbf{x}_j)}$.

Gradient Descent (GD) Algorithm

Logistic regression: $\min_{\mathbf{w}} \frac{1}{n} \sum_{j=1}^n l(y_j \mathbf{w}^T \mathbf{x}_j)$, where $l(z) = \log(1 + e^{-z})$.

$$\text{Gradient at } \mathbf{w}_t: \mathbf{g}_t = \frac{1}{n} \sum_{j=1}^n \frac{-y_j \mathbf{x}_j}{1 + \exp(y_j \mathbf{w}_t^T \mathbf{x}_j)}.$$

GD repeat:

1. Compute gradient: $\mathbf{g}_t$
2. Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \mathbf{g}_t$



Tune the step size (learning rate) α

Algorithms

Gradient Descent (GD)

Accelerated GD

Stochastic GD

AGD Algorithm

Logistic regression: $\min_{\mathbf{w}} \frac{1}{n} \sum_{j=1}^n l(y_j \mathbf{w}^T \mathbf{x}_j)$, where $l(z) = \log(1 + e^{-z})$.

$$\text{Gradient at } \mathbf{w}_t: \mathbf{g}_t = \frac{1}{n} \sum_{j=1}^n \frac{-y_j \mathbf{x}_j}{1 + \exp(y_j \mathbf{w}_t^T \mathbf{x}_j)}.$$

AGD repeat:

1. Compute gradient: $\mathbf{g}_t$
2. Update momentum: $\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \mathbf{g}_t$
3. Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \mathbf{v}_{t+1}$

Tune α and β ($0 \leq \beta < 1$)

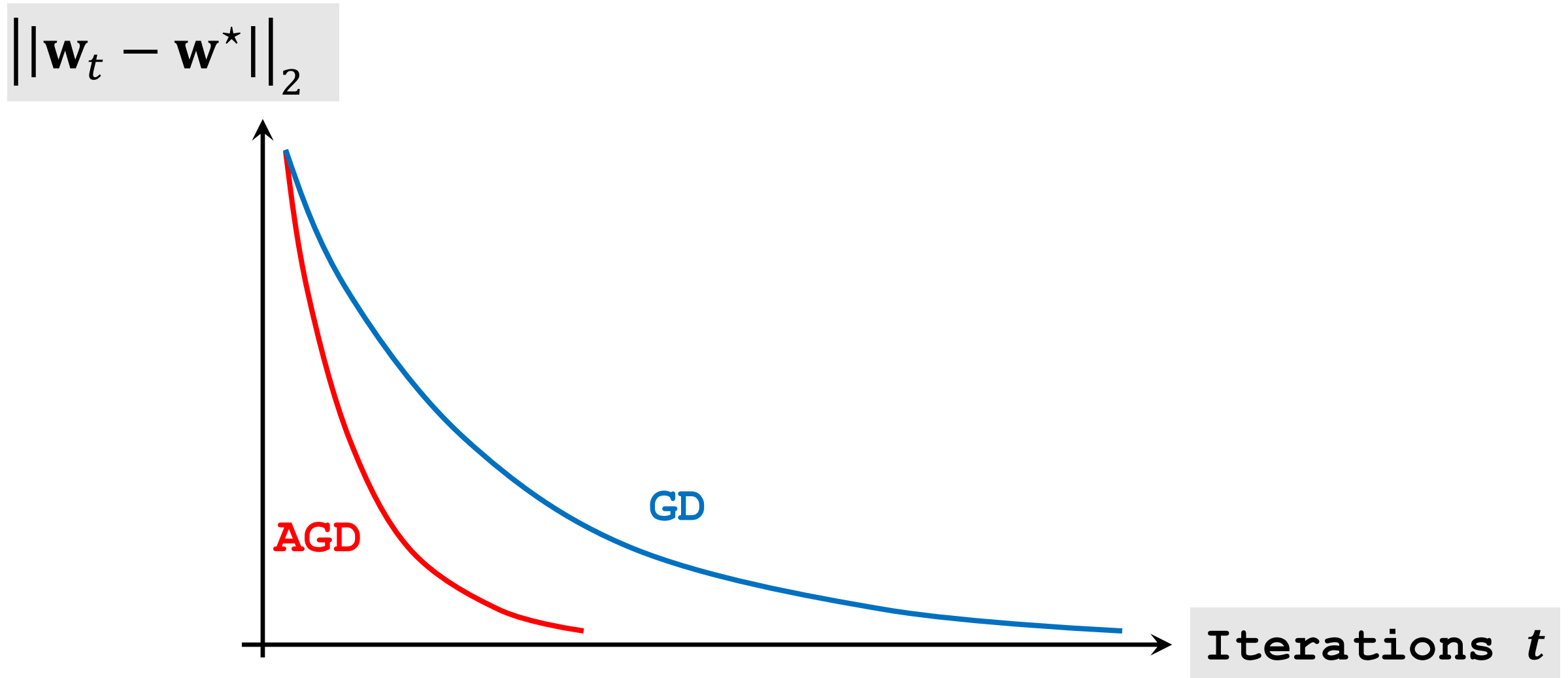
Algorithms

Gradient Descent (GD)

Accelerated GD

Stochastic GD

GD versus AGD



Time Complexity

$$\text{Gradient at } \mathbf{w}_t: \mathbf{g}_t = \frac{1}{n} \sum_{j=1}^n \tilde{\mathbf{g}}_{t,j}, \quad \text{where } \tilde{\mathbf{g}}_{t,j} = \frac{-y_j \mathbf{x}_j}{1 + \exp(y_j \mathbf{w}_t^T \mathbf{x}_j)}.$$

Per-iteration time complexity is $O(nd)$.

- $O(d)$ time for computing $\mathbf{w}_t^T \mathbf{x}_j$.
- $O(d)$ time for computing $\tilde{\mathbf{g}}_{t,j}$.
- $O(nd)$ time for computing all the $\tilde{\mathbf{g}}_{t,j}$.

Algorithms

Gradient Descent (GD)

Accelerated GD

Stochastic GD

SGD Algorithm

$$\text{Gradient at } \mathbf{w}_t: \mathbf{g}_t = \frac{1}{n} \sum_{j=1}^n \tilde{\mathbf{g}}_{t,j}, \quad \text{where } \tilde{\mathbf{g}}_{t,j} = \frac{-y_j \mathbf{x}_j}{1 + \exp(y_j \mathbf{w}_t^T \mathbf{x}_j)}.$$

The stochastic gradient is close to the full gradient:

$$\mathbf{g}_t = \mathbb{E}_j[\tilde{\mathbf{g}}_{t,j}],$$

where j is randomly sampled from $\{1, \dots, n\}$.

Algorithms

Gradient Descent (GD)

Accelerated GD

Stochastic GD

SGD Algorithm

$$\text{Gradient at } \mathbf{w}_t: \mathbf{g}_t = \frac{1}{n} \sum_{j=1}^n \tilde{\mathbf{g}}_{t,j}, \quad \text{where } \tilde{\mathbf{g}}_{t,j} = \frac{-y_j \mathbf{x}_j}{1 + \exp(y_j \mathbf{w}_t^T \mathbf{x}_j)}.$$

SGD repeats

1. Randomly draw j from $\{1, 2, \dots, n\}$.
2. Compute the stochastic gradient $\tilde{\mathbf{g}}_{t,j}$.
3. Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \tilde{\mathbf{g}}_{t,j}$.

Per-iteration time complexity is $O(d)$.

Algorithms

Gradient Descent (GD)

Accelerated GD

Stochastic GD

Accelerated SGD Algorithm

$$\text{Gradient at } \mathbf{w}_t: \mathbf{g}_t = \frac{1}{n} \sum_{j=1}^n \tilde{\mathbf{g}}_{t,j}, \quad \text{where } \tilde{\mathbf{g}}_{t,j} = \frac{-y_j \mathbf{x}_j}{1 + \exp(y_j \mathbf{w}_t^T \mathbf{x}_j)}.$$

Accelerated SGD repeats

1. Randomly draw j from $\{1, 2, \dots, n\}$.
2. Compute the stochastic gradient $\tilde{\mathbf{g}}_{t,j}$.
3. Update momentum: $\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \tilde{\mathbf{g}}_{t,j}$.
4. Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \mathbf{v}_{t+1}$.

Algorithms

Gradient Descent (GD)

Accelerated GD

Stochastic GD

SGD Algorithm

$$\text{Gradient at } \mathbf{w}_t: \mathbf{g}_t = \frac{1}{n} \sum_{j=1}^n \tilde{\mathbf{g}}_{t,j}, \quad \text{where } \tilde{\mathbf{g}}_{t,j} = \frac{-y_j \mathbf{x}_j}{1 + \exp(y_j \mathbf{w}_t^T \mathbf{x}_j)}.$$

Output of SGD:

- Option 1: output the last iteration $\mathbf{w}_{T+1}$
- Option 2: output the average of $\mathbf{w}$ produced by the last tens of iteration.

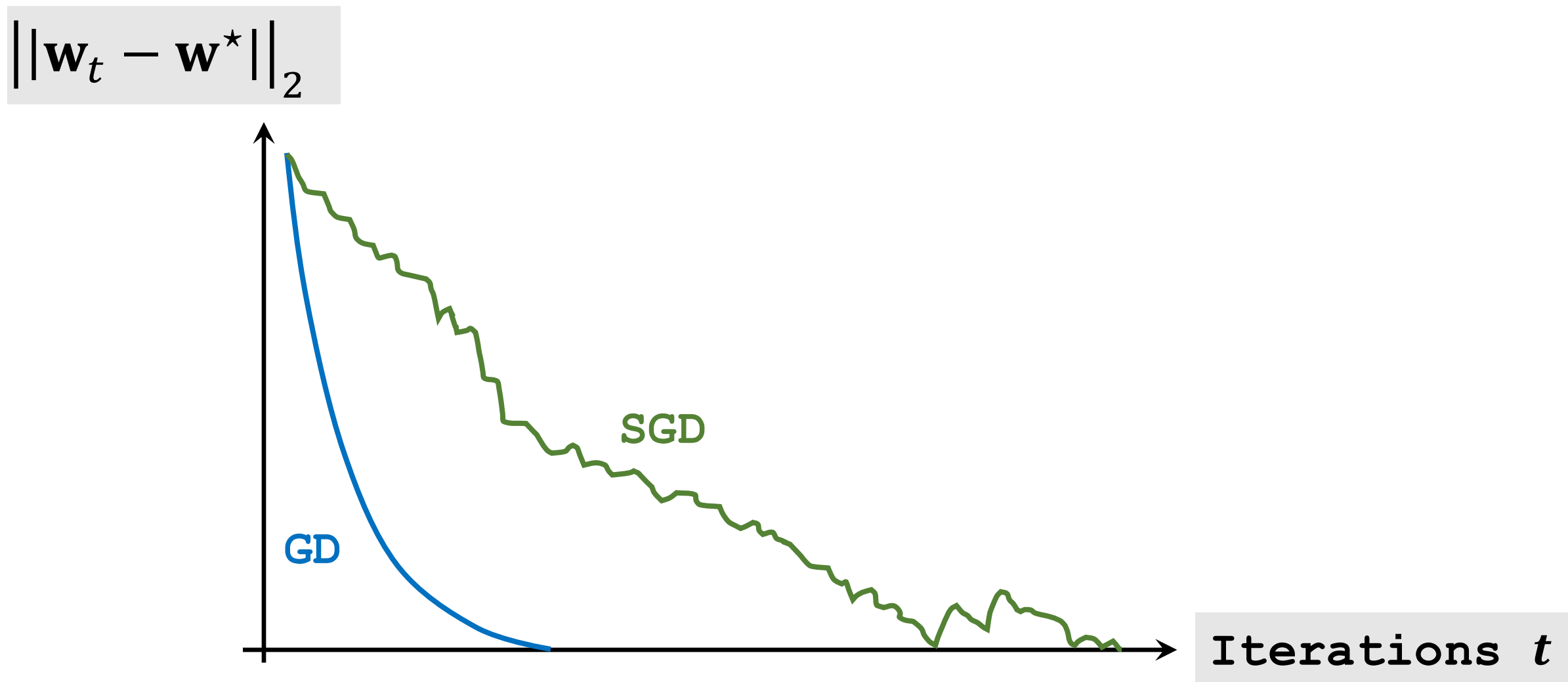
Algorithms

Gradient Descent (GD)

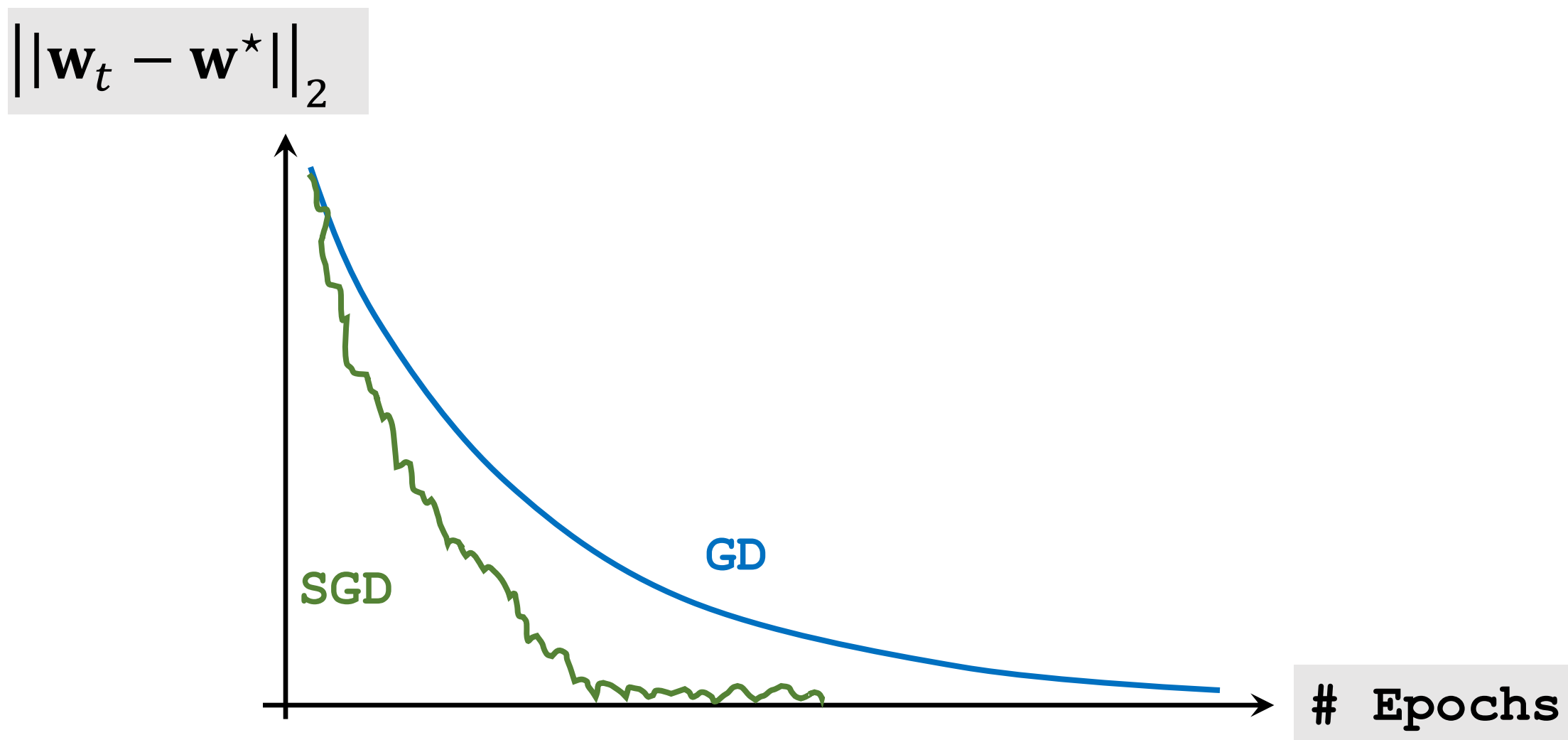
Accelerated GD

Stochastic GD

GD versus SGD



GD versus SGD



Training and Prediction

- Training:

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} \sum_j l(y_j \mathbf{w}^T \mathbf{x}_j), \text{ where } l(z) = \log(1 + e^{-z}).$$

- For a test feature vector $\mathbf{x}' \in \mathbb{R}^d$, make prediction by $\operatorname{sign}(\mathbf{x}'^T \mathbf{w}^*)$.

Summary

- Logistic regression model for *linear binary* classification.

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} \sum_j l(y_j \mathbf{w}^T \mathbf{x}_j), \text{ where } l(z) = \log(1 + e^{-z}).$$

- Compute the gradient using vector derivatives and the chain rule.
- Gradient-based algorithms: GD, AGD, SGD, etc.
- Make prediction using $\operatorname{sign}(\mathbf{x}'^T \mathbf{w}^*)$.

Evaluate Binary Classification

Evaluate Binary Classification

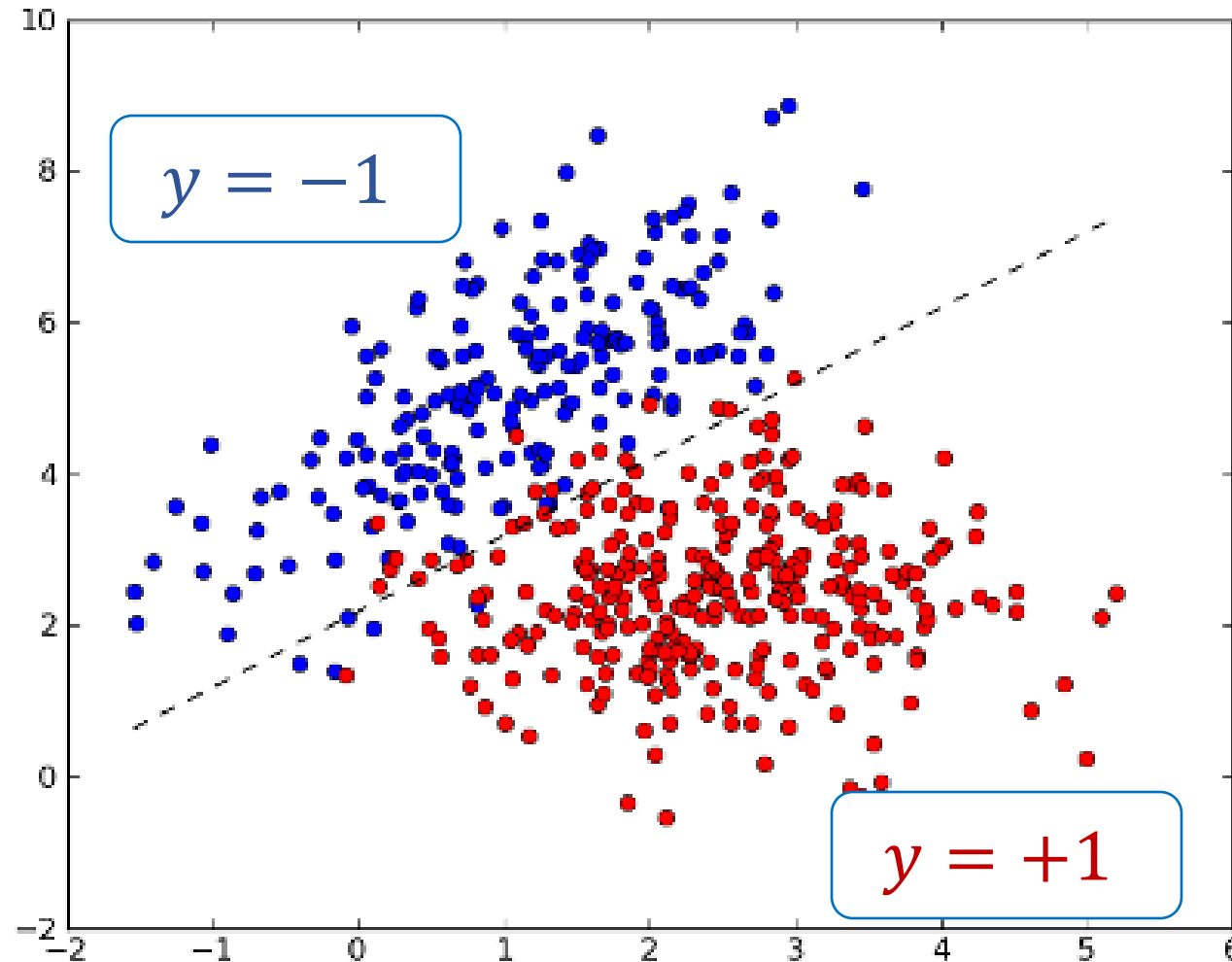
- Error Rate = $\frac{\text{\# Classification Errors}}{\text{\# Samples}}$
- Accuracy = 1 - Error Rate

Evaluate Binary Classification

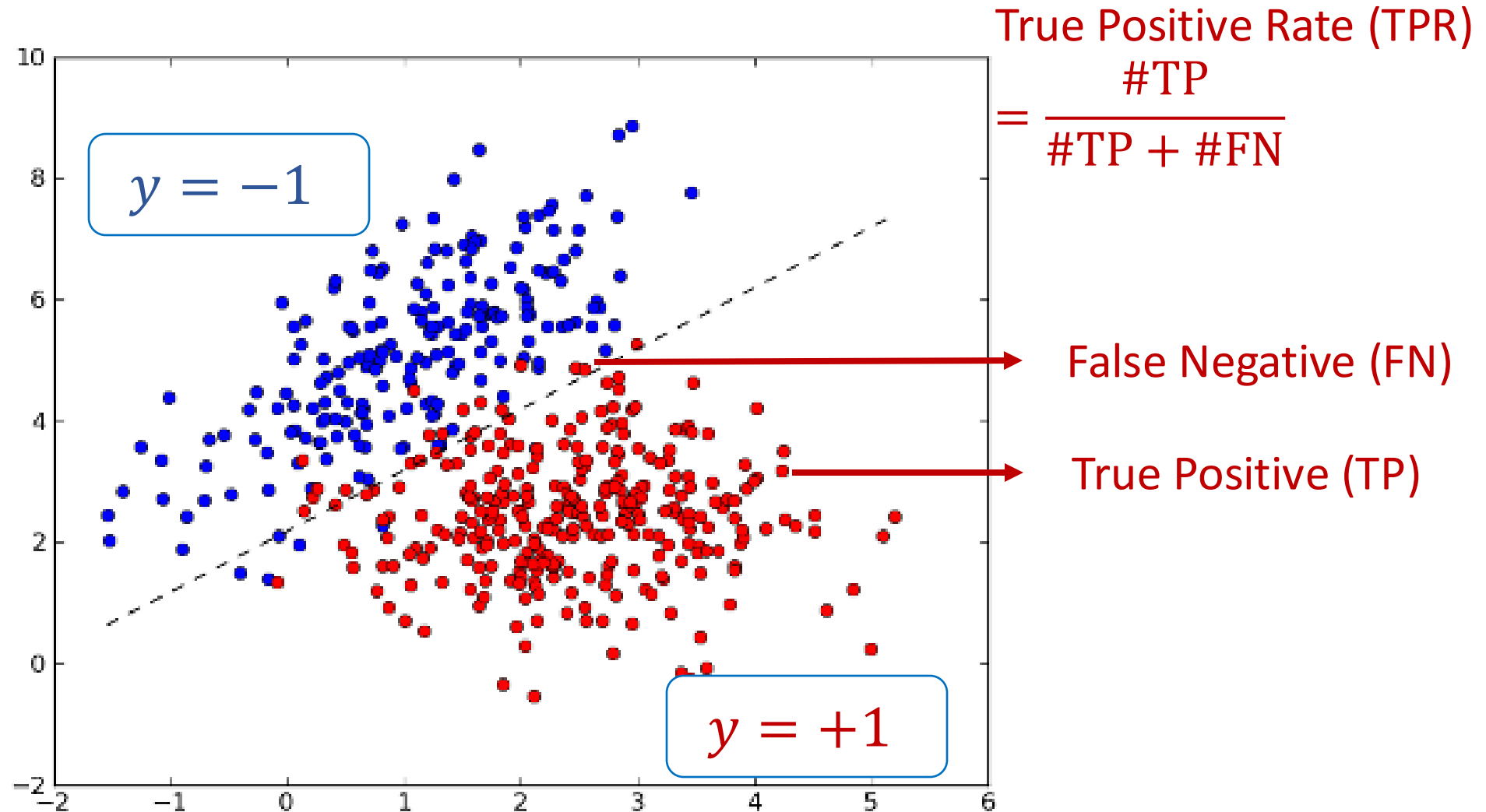
- Error Rate = $\frac{\text{\# Classification Errors}}{\text{\# Samples}}$
- Accuracy = 1 - Error Rate

Error rate and **Accuracy** are not meaningful in class-imbalanced problems.

Evaluate Binary Classification



Evaluate Binary Classification



Evaluate Binary Classification

False Positive Rate (FPR)

$$= \frac{\#FP}{\#FP + \#TN}$$

True Positive Rate (TPR)

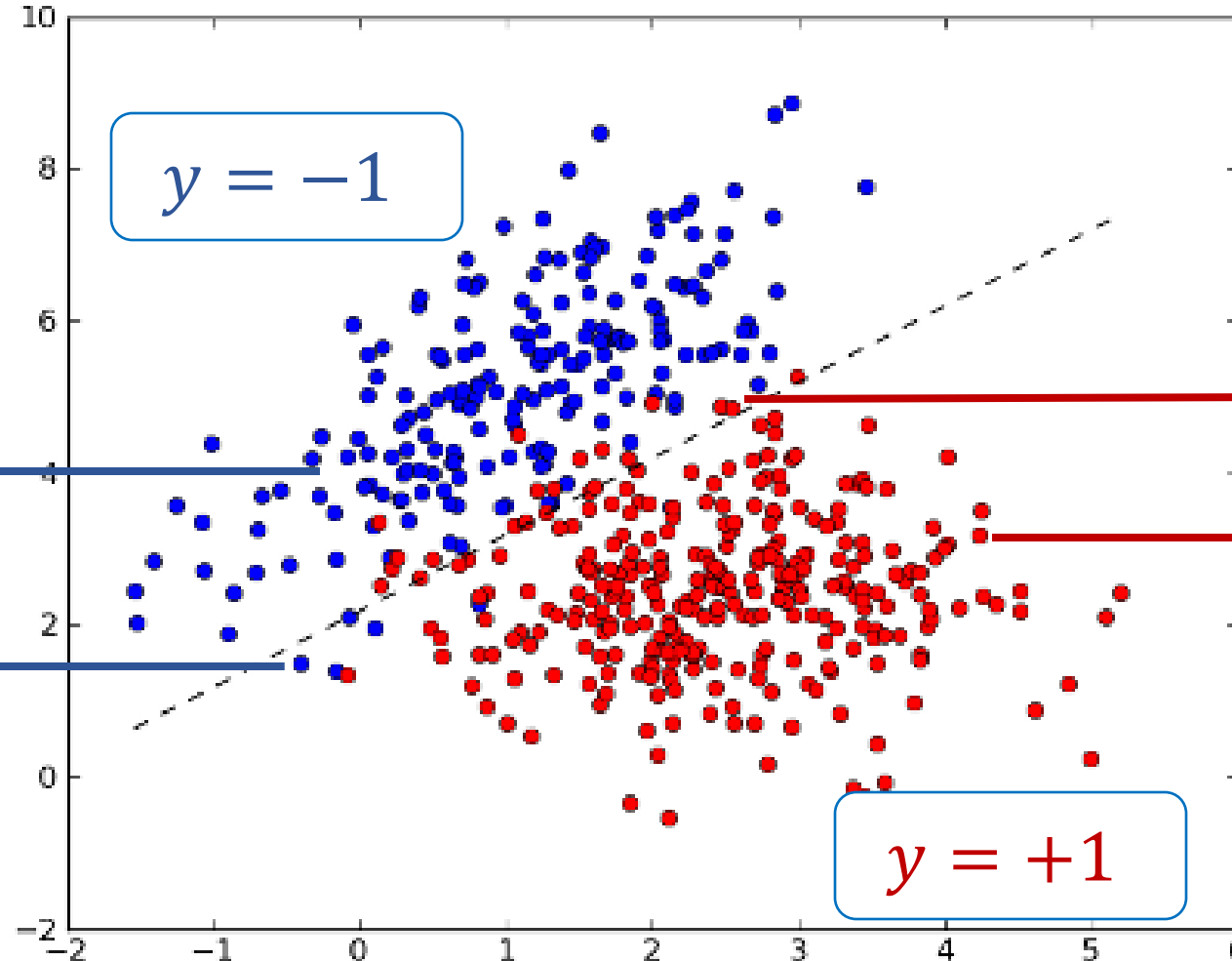
$$= \frac{\#TP}{\#TP + \#FN}$$

True Negative (TN)

False Positive (FP)

False Negative (FN)

True Positive (TP)



Evaluate Binary Classification

False Positive Rate (FPR)

$$= \frac{\#FP}{\#FP + \#TN}$$

True Negative (TN)

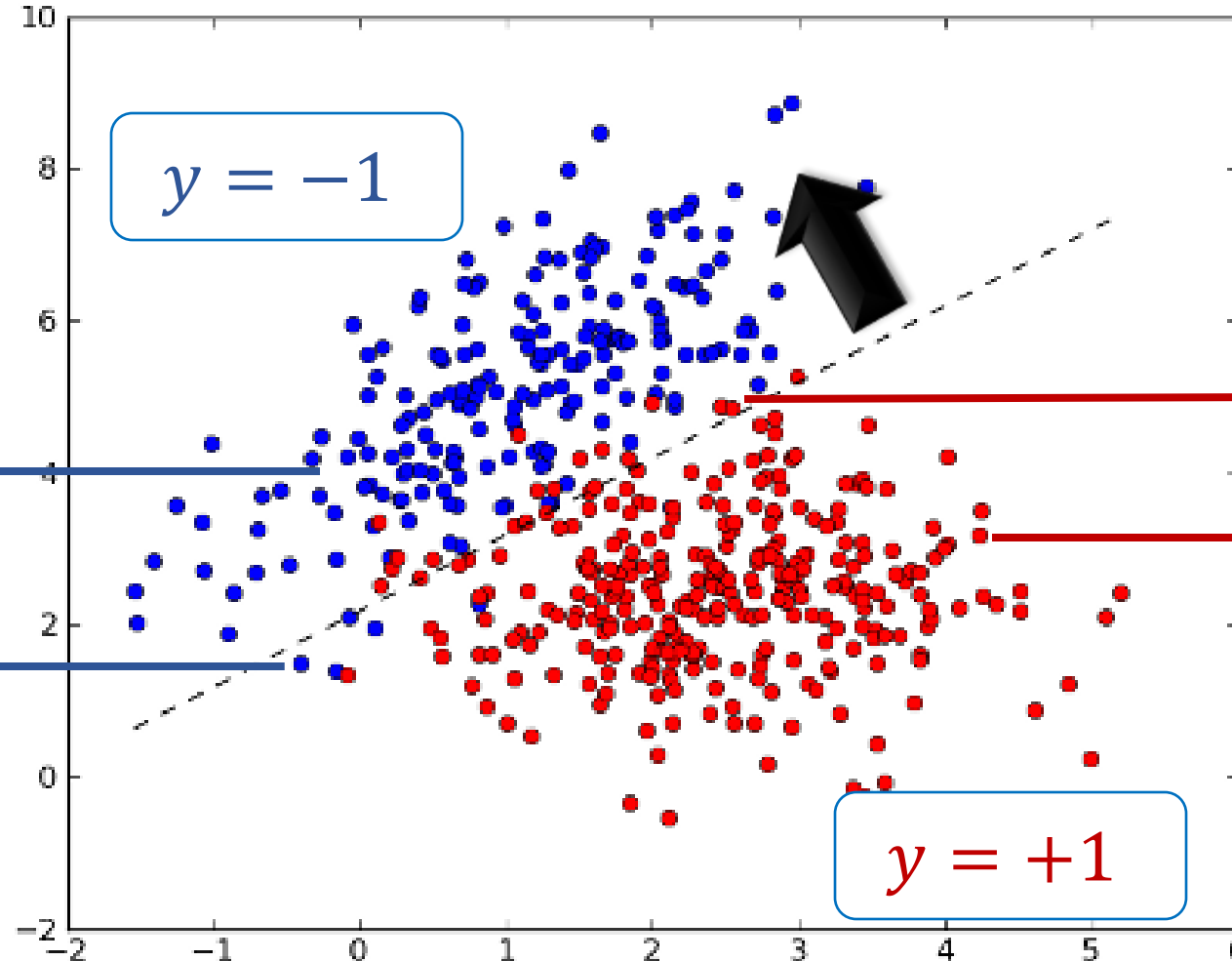
False Positive (FP)

True Positive Rate (TPR)

$$= \frac{\#TP}{\#TP + \#FN}$$

False Negative (FN)

True Positive (TP)



Evaluate Binary Classification

False Positive Rate (FPR)

$$= \frac{\#FP}{\#FP + \#TN}$$

Healthy (negative)

$$y = -1$$

True Positive Rate (TPR)

$$= \frac{\#TP}{\#TP + \#FN}$$

True Negative (TN)

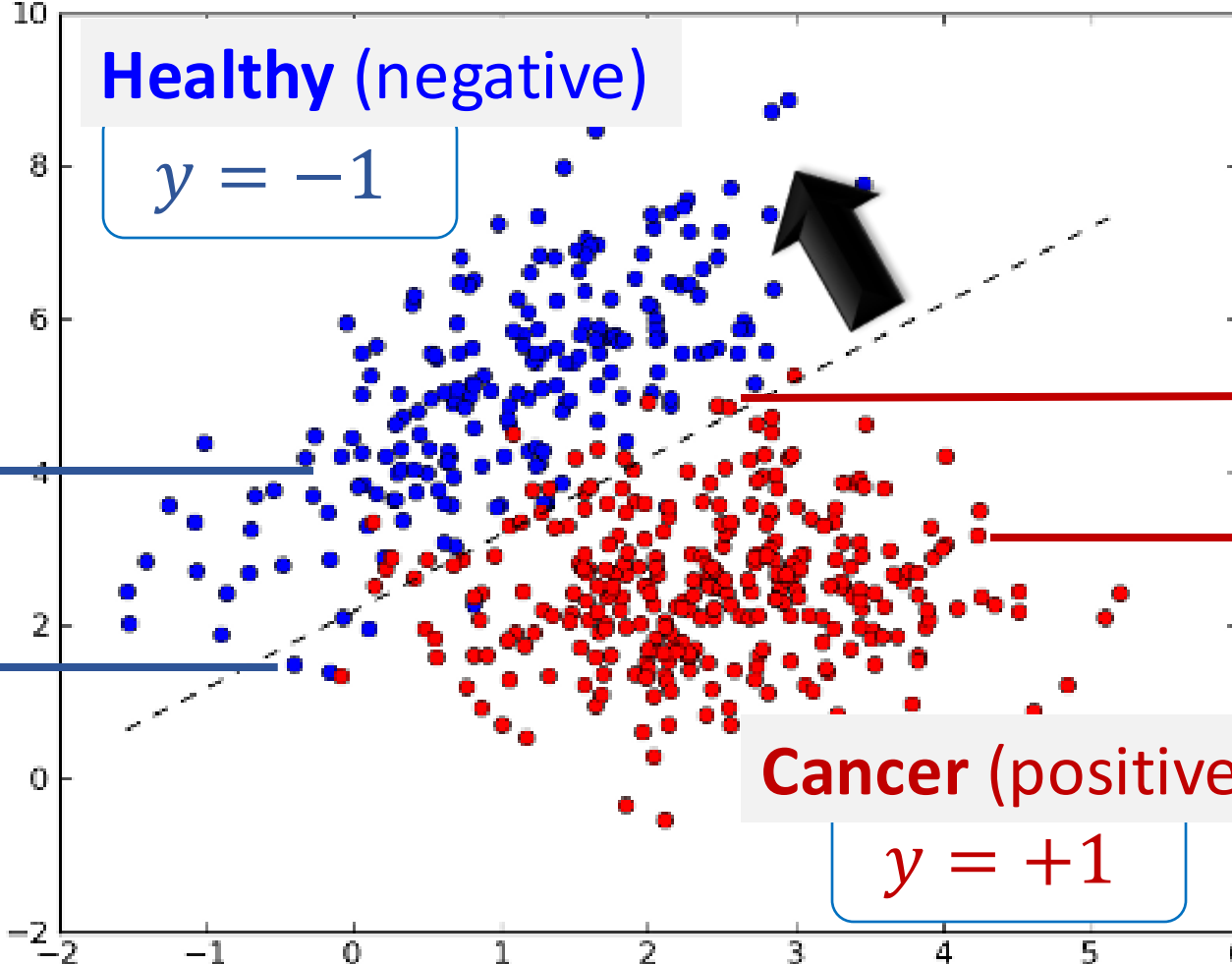
False Positive (FP)

False Negative (FN)

True Positive (TP)

Cancer (positive)

$$y = +1$$



Evaluate Binary Classification

False Positive Rate (FPR)

$$= \frac{\#FP}{\#FP + \#TN}$$

True Positive Rate (TPR)

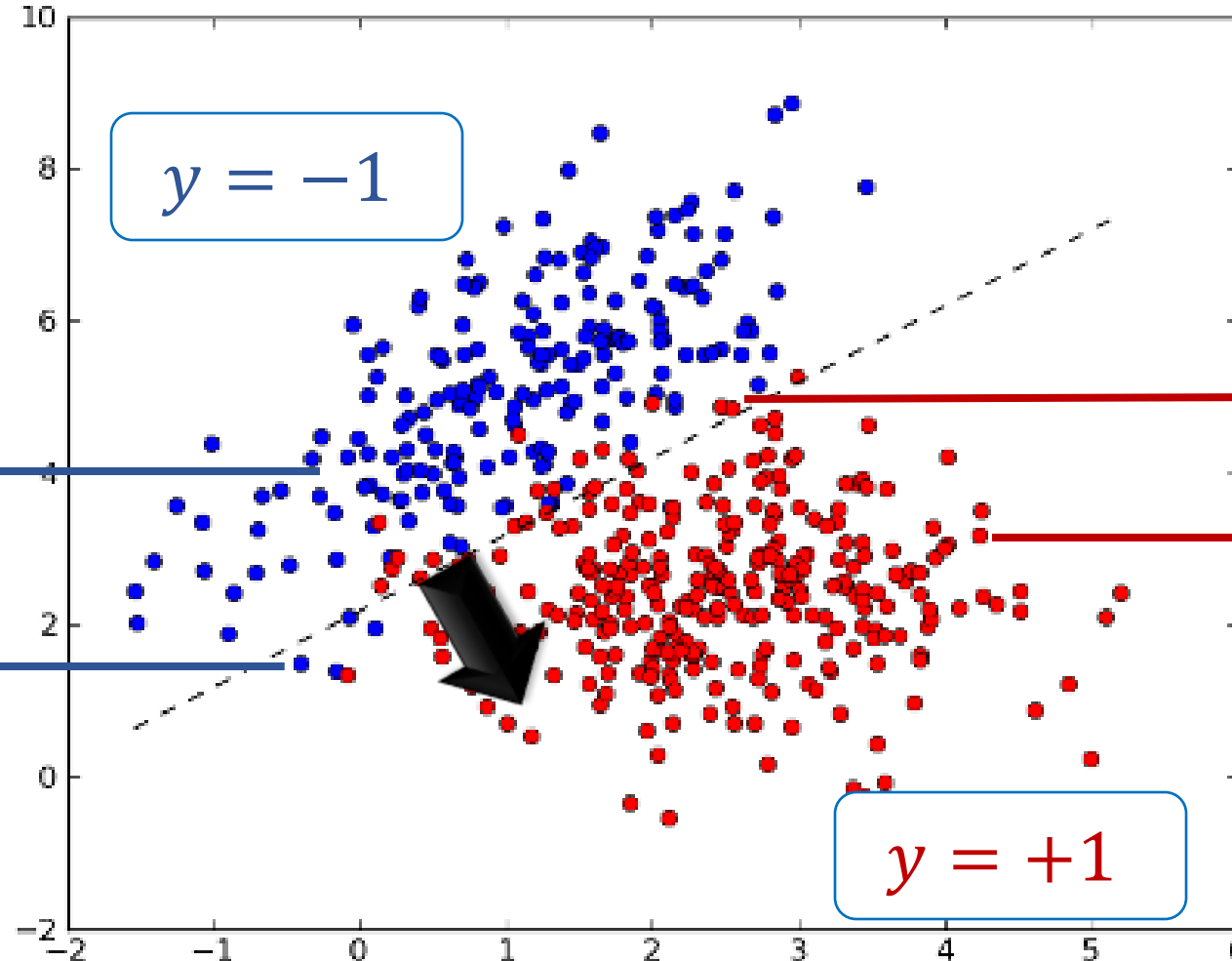
$$= \frac{\#TP}{\#TP + \#FN}$$

True Negative (TN)

False Positive (FP)

False Negative (FN)

True Positive (TP)



Evaluate Binary Classification

False Positive Rate (FPR)

$$= \frac{\#FP}{\#FP + \#TN}$$

Non-spam (negative)

$$y = -1$$

True Positive Rate (TPR)

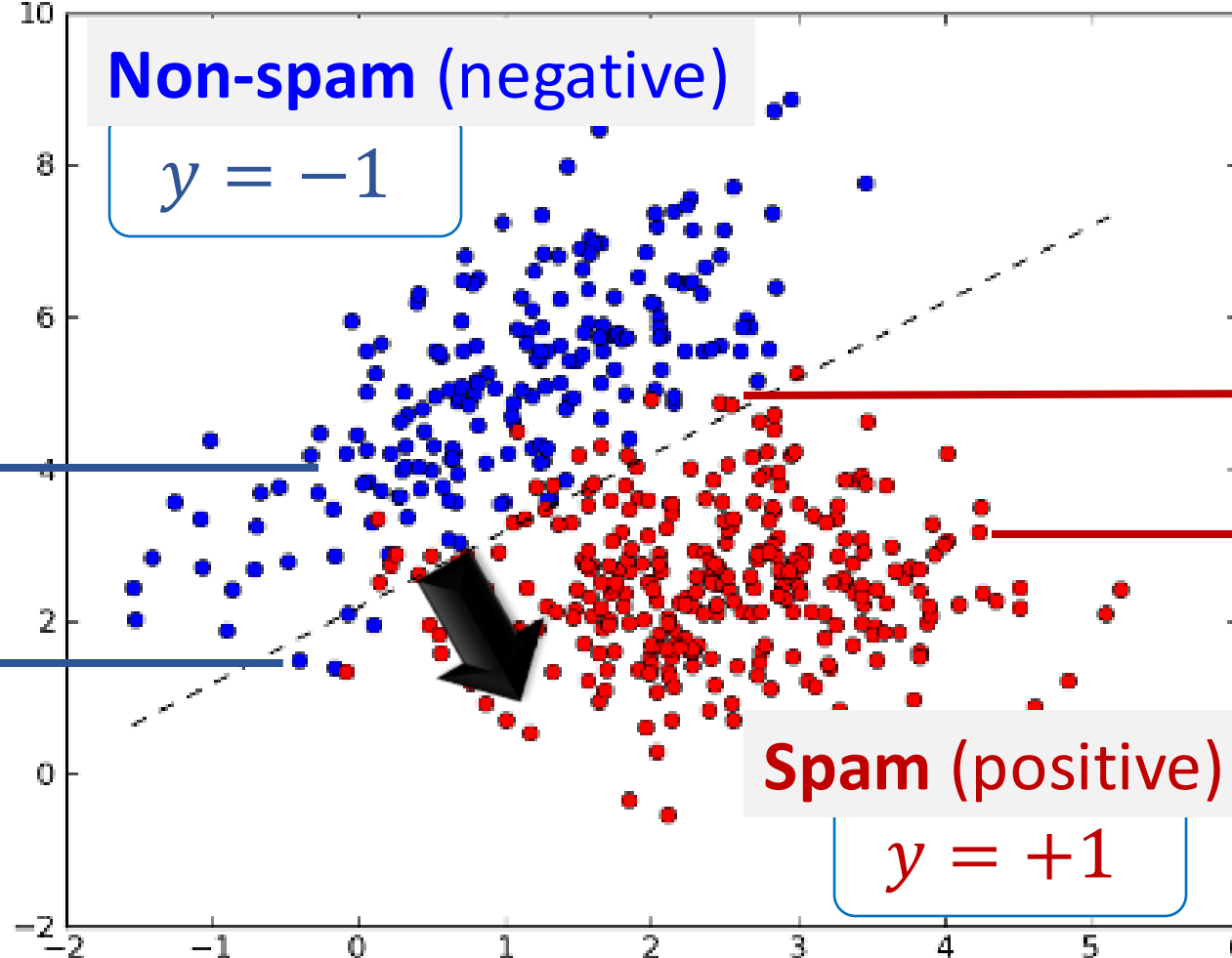
$$= \frac{\#TP}{\#TP + \#FN}$$

True Negative (TN)

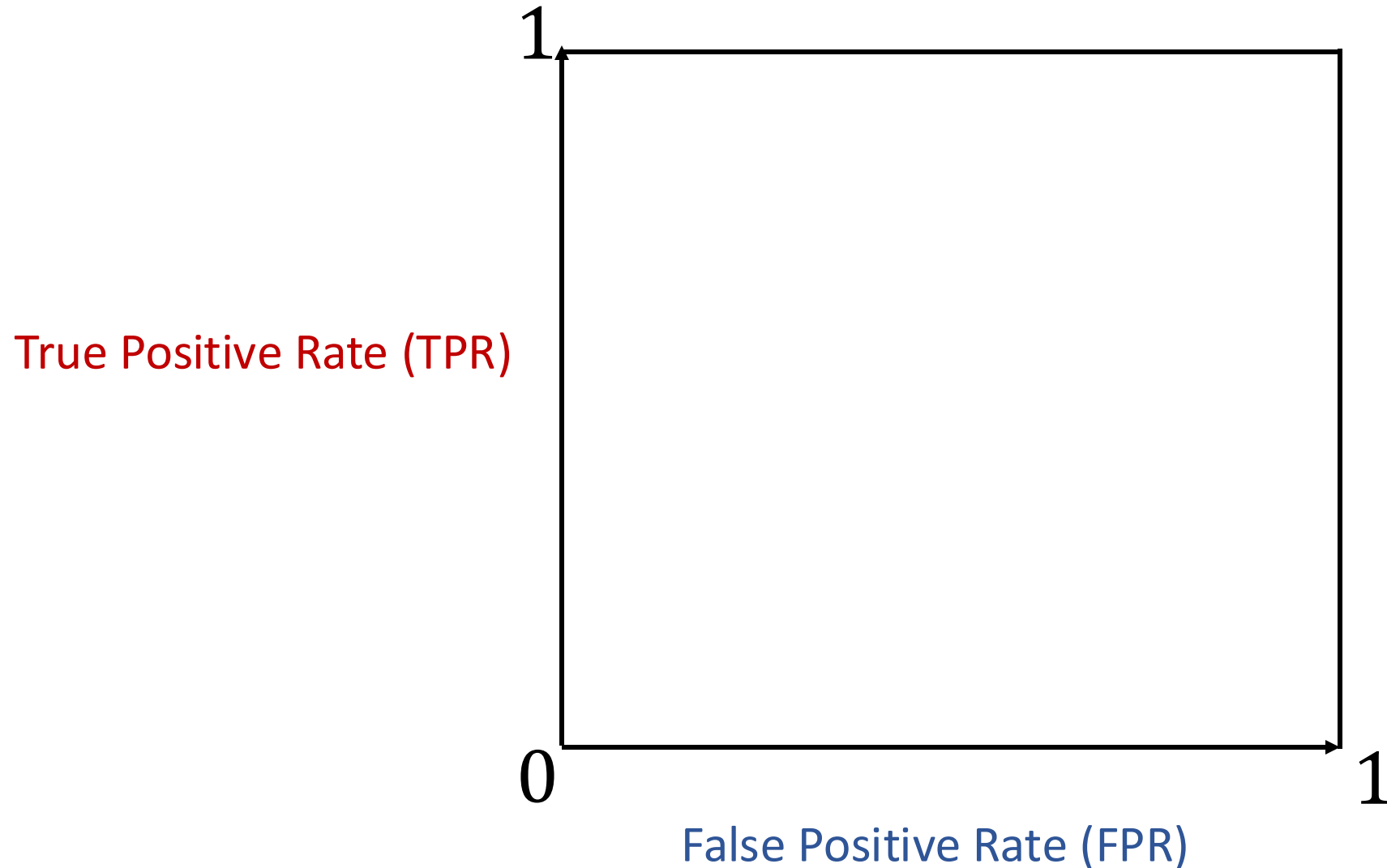
False Positive (FP)

False Negative (FN)

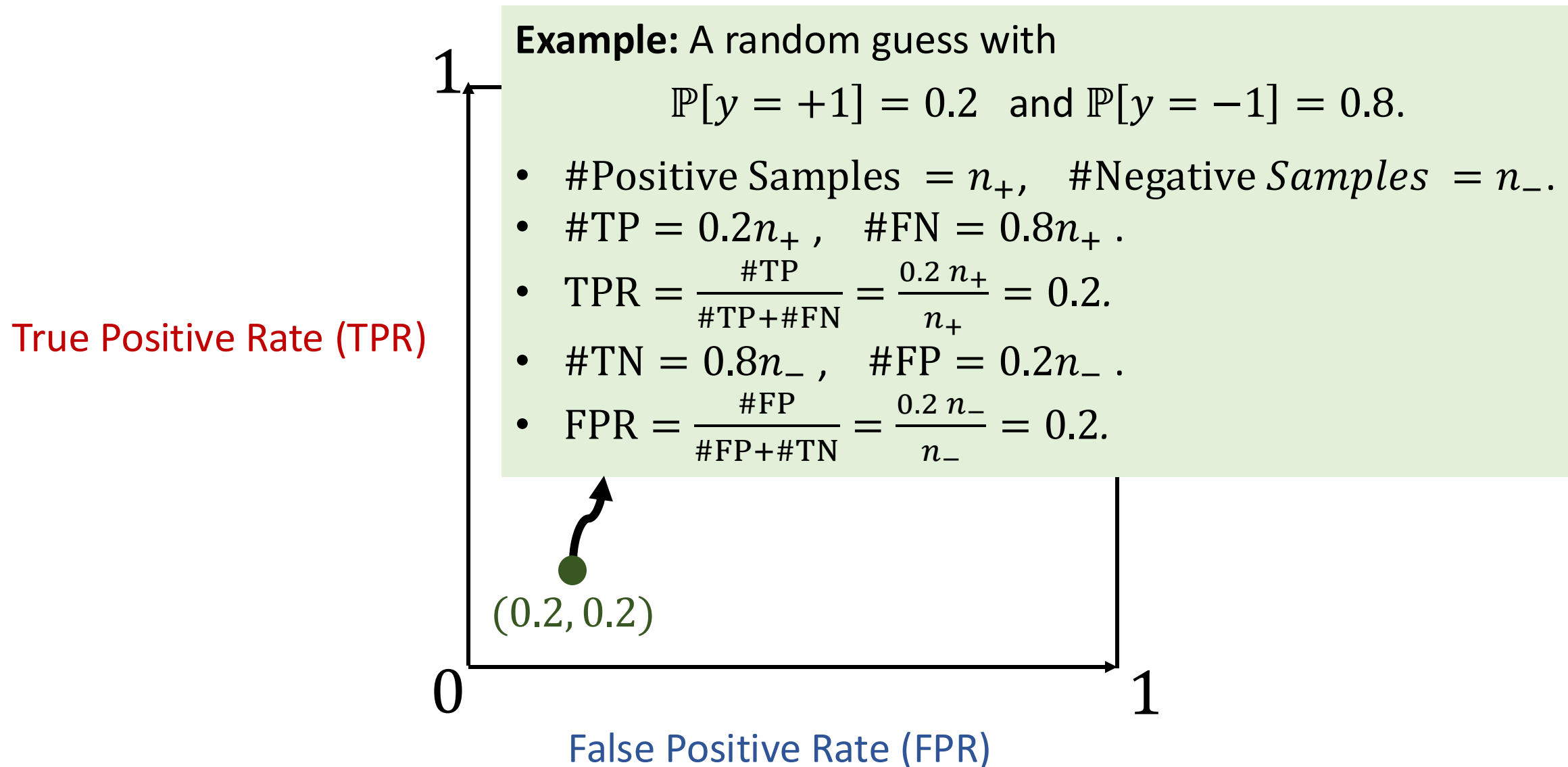
True Positive (TP)



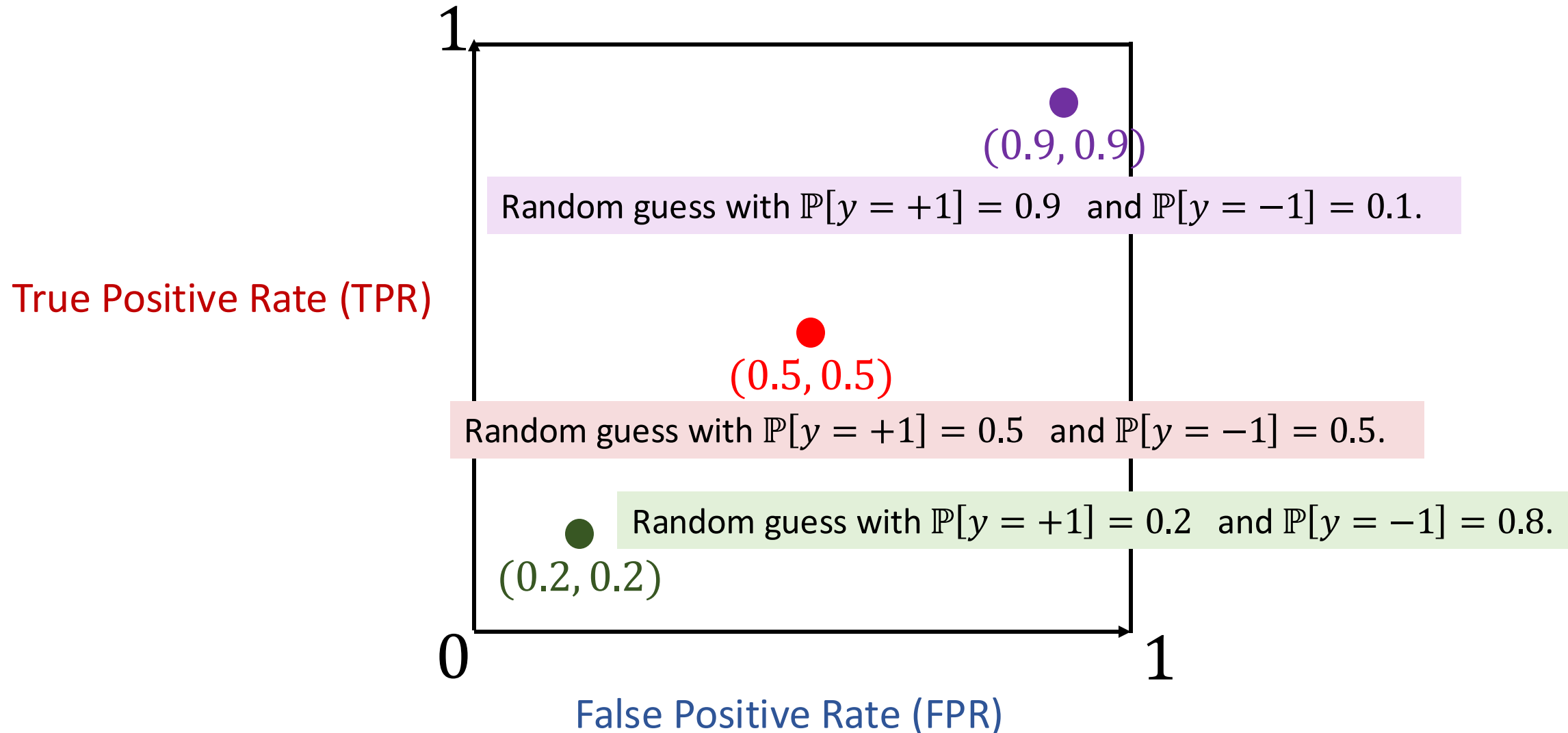
Receiver Operating Characteristic (ROC) Curve



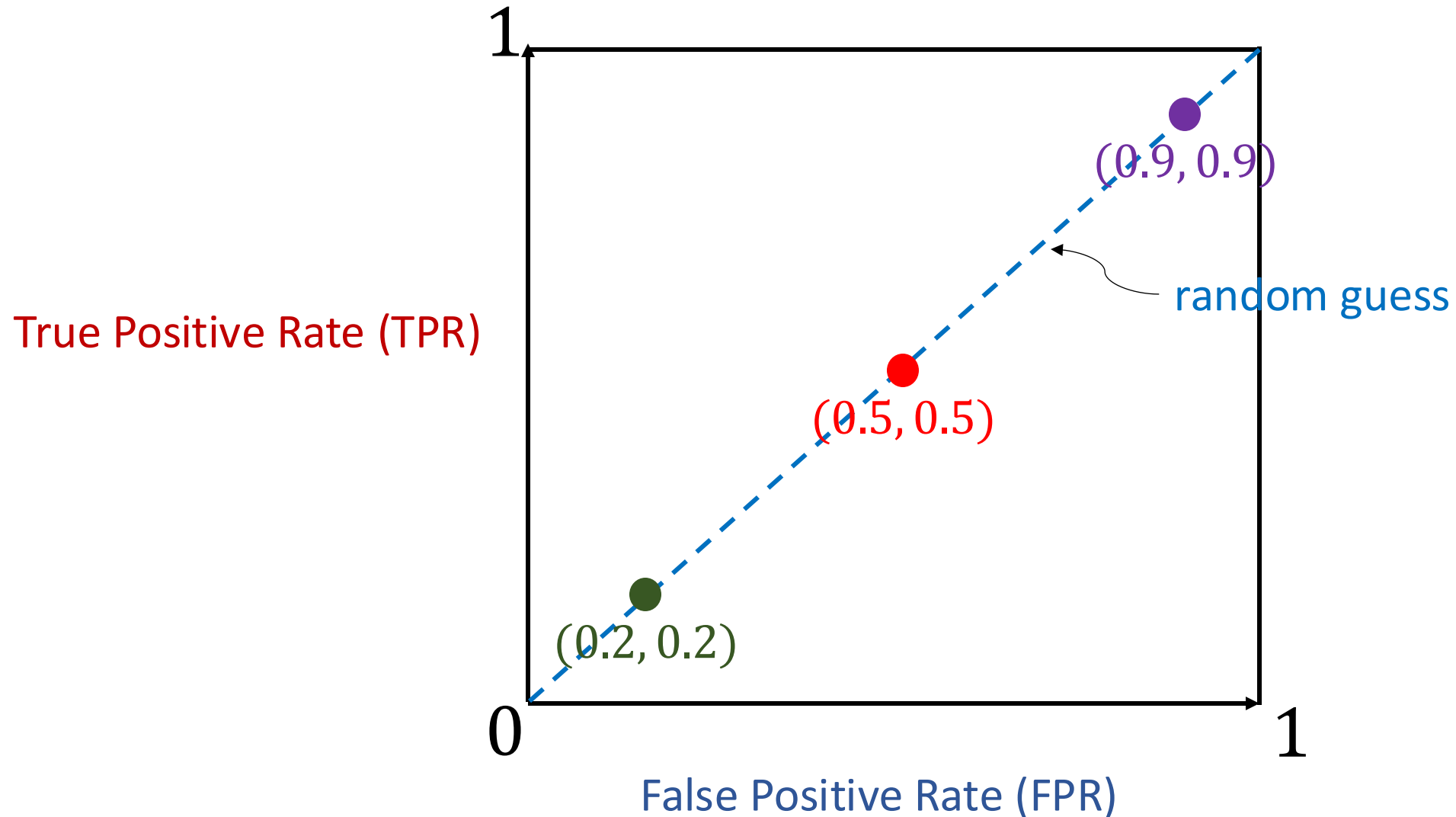
Receiver Operating Characteristic (ROC) Curve



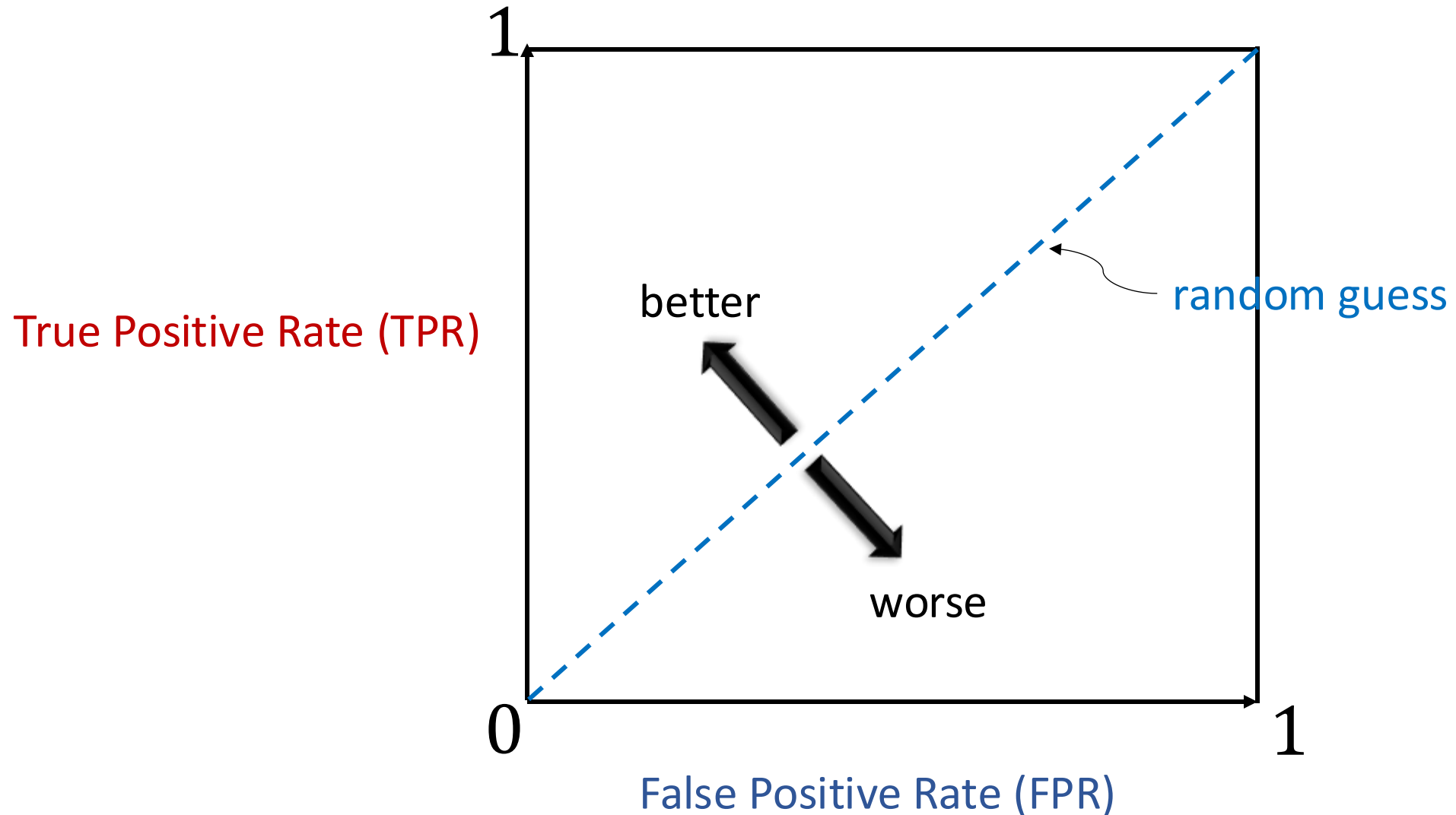
Receiver Operating Characteristic (ROC) Curve



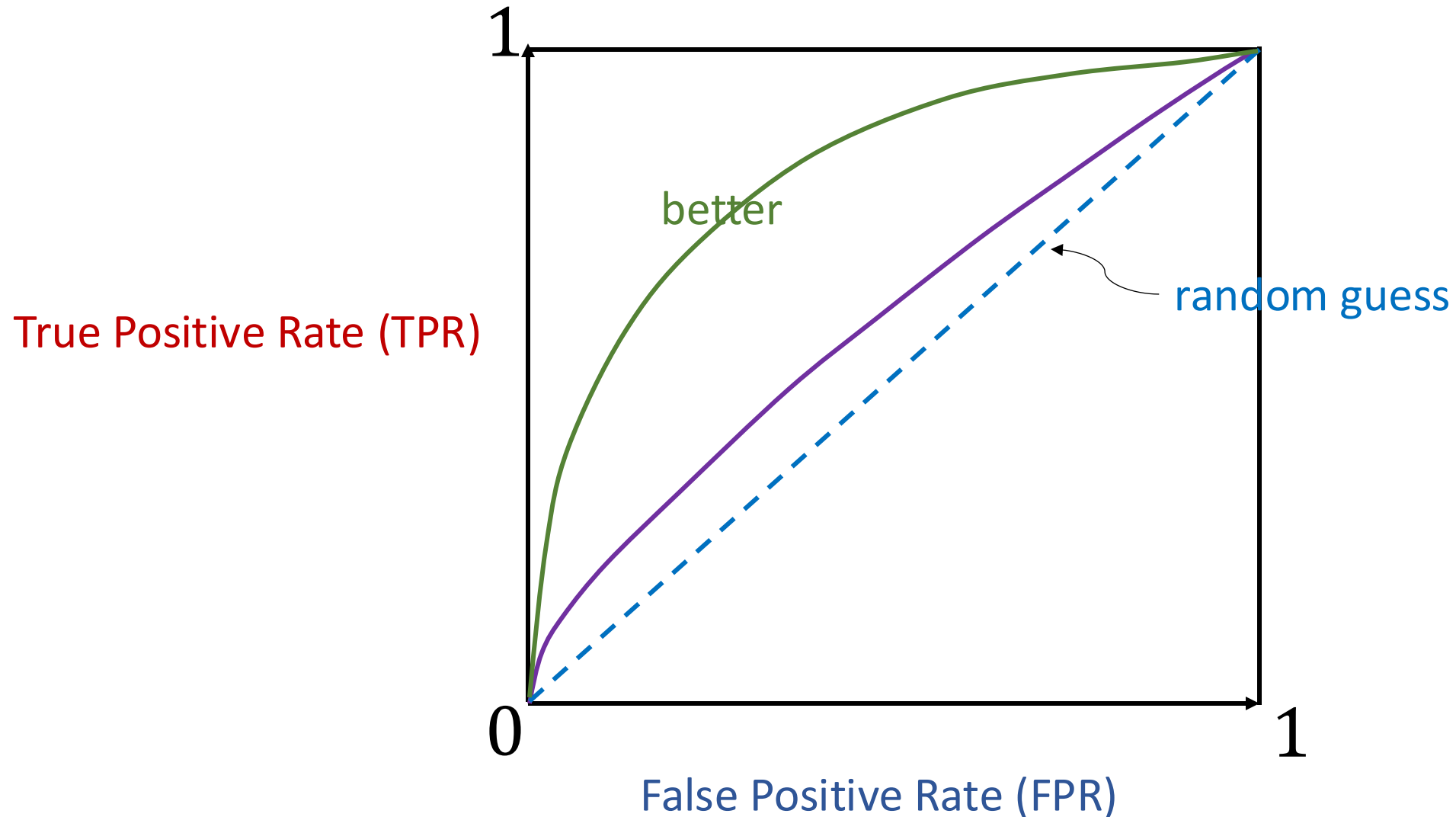
Receiver Operating Characteristic (ROC) Curve



Receiver Operating Characteristic (ROC) Curve



Receiver Operating Characteristic (ROC) Curve



Thank you!